

Afillar Insurance APIs

Developer Guide

*Architecture, code, configuration, workflow, portal and operations: the hand-over reference
for maintainers*

Version 1.2

05-Oct-2026

Prepared for the Afillar Channel Integration Program

Table of Contents

Table of Contents	2
1. Introduction	4
1.1 Purpose of this Guide	4
1.2 What the Solution Is	4
1.3 Glossary	4
1.4 Read This First: What Will Surprise You	5
1.5 Release History	6
2. Architecture	7
2.1 Layers	7
2.2 Design Rules	7
2.3 Request Lifecycle	8
3. Source Layout and Tooling	9
3.1 Workspace Layout	9
3.2 The imo Tool	9
3.3 Groovy Rules of This Runtime	9
3.4 Deployment Order of the Code	10
4. Code Reference	11
4.1 Asset Inventory	11
4.2 Common Functions	12
4.3 APIs	20
5. API Walkthroughs	24
6. Data and Configuration Reference	26
6.1 Tables	26
6.2 Configuration Keys (UP_ApiConfig)	27
6.3 Field Rules (UP_FieldRule)	30
6.4 Security Baselines (UPSecurity)	31
7. Platform Configuration (Products)	32
7.1 What Exists on the Platform	32
7.2 Data Dictionary Bindings	32
7.3 How the Products Were Built	32
8. Business Algorithms	33
8.1 Premium Mode, Term and Age	33
8.2 Cancellation (Checker / Approver)	33
8.3 Platform Quirks That Shaped the Code	34
9. Errors, Logging and Tracing	35
10. How-To Recipes	36
11. Test Tooling	38
12. Test Procedure for a Change	40
13. Environments and Deployment	41
14. Known Limitations and Backlog	42
14.1 Troubleshooting	42
14.2 First-Day Checklist for a New Developer	42
15. Workflow, Roles and Dashboard	43
15.1 Overview	43
15.2 Roles and Identity	43
15.3 The Maker Stamp (AgentCode)	44
15.4 Queues	44
15.5 Dashboard	45
15.6 Maker-Checker Rule and the Test Switch	45

15.7 New API Contracts (examples)	45
16. Notifications (Email and SMS)	47
16.1 Design	47
16.2 Events and Templates	47
16.3 Modes and Result Status	47
16.4 SNS Integration Details	47
16.5 Switching to Live Delivery	48
17. The Portal (UIC Page)	49
17.1 What and Where	49
17.2 Structure of the File	49
17.3 Sign-In and Session	51
17.4 API Access	51
17.5 Configuration and Branding	52
17.6 Deploy and Test the Portal	52
17.7 Portal Limitations	52
18. PA001, iHub and the Other Assets in the Tenant	53
18.1 What Exists	53
18.2 How the PA001 APIs Work	53
18.3 Comparison	54
18.4 What Is Not Known	54
18.5 If iHub Is Wanted Later	54
19. Operations Runbook and Handover Checklist	55
19.1 Switches and How to Change Them	55
19.2 Users and Roles	55
19.3 Troubleshooting the v1.2 Features	55
19.4 Go-Live Checklist for the v1.2 Additions	56
19.5 What a New Developer Needs	56
19.6 Backlog (Consolidated)	56
Appendix A. File Inventory	58
Appendix B. Table Definitions	59

1. Introduction

1.1 Purpose of this Guide

This guide is for the developers who will maintain and extend the Afillar Insurance Company insurance APIs after hand-over. It explains how the solution is built, where every piece of logic and data lives, how a request travels through the code, which platform services are called and why, which platform quirks shaped the design, how to change or add things safely, how to test, and how to deploy. The reference sections (4, 6, 11, 12 and the appendices) are generated from the real source files, tables and scripts, so they match what is deployed in the MC tenant at the time of writing. Version 1.2 adds the maker-checker workflow, the dashboard, notifications, the staff portal and an analysis of PA001 and iHub (sections 15 to 19). **Section 1.4 lists what is most likely to surprise a new developer: read it first.**

1.2 What the Solution Is

A set of 18 active REST APIs (and 2 inactive ones: a retired stub and a test probe) with 13 shared functions, written as iComposer Groovy scripts on the InsureMO platform, plus a staff portal (a UIC page). They orchestrate the platform's proposal, policy, endorsement, search and print services for seven insurance products (UPPA001 Personal Accident, TL001 Term Life, FP001 Female Protection, HC001 Hospital Cash, CI001 Critical Illness, LP001 Life Protect, DH001 Domestic Helper; 11 variants, 57 plans). Partners such as Sohar Bank call the APIs; the APIs validate the request, call the platform and return a stable contract. Bank staff work in three roles (Maker, Proposal Checker, Cancellation Checker) through the portal. Everything that a business user may want to change (validation rules, product data, refund rules, texts, formats, channels, insurers, users and roles, message texts) is data in iTables, not code. The solution is developed in iComposer only: iHub is not used (section 18).

1.3 Glossary

Term	Meaning
MC	The current development tenant in which everything is built and tested (tenant code <code>uponly</code> , gateway portal-gw.insuremo.com). UAT is a separate tenant.
iComposer	InsureMO's low-code Groovy runtime. An API script is exposed as a REST route; a function (common service) is a shared Groovy class called with <code>getCommonService</code> .
iTables / data table	Tenant configuration tables read by the code (<code>UP_ApiConfig</code> , <code>UP_FieldRule</code> , ...).
Tech product / market product	The platform keeps a technical product (<code><CODE>_TECH</code> , holds the data dictionary tree) and a market product (<code><CODE></code>) per product code.
DD / binding	Data dictionary: the fields of the policy tree. A field must be bound to an object before the platform accepts or calculates it.
Maker	Bank employee who creates quotations and proposals and raises cancellation requests.
Proposal Checker	Bank employee who approves (issues) or rejects a submitted proposal.
Cancellation Checker	Bank employee who approves or rejects a cancellation request (named Approver in v1.0 of the cancellation API).
AgentCode	Platform proposal field in which the maker's platform user id is stored (section 15.3).
SNS	The platform's notification service used to send email and SMS (section 16).
UIC	InsureMO UI Connector: hosts the portal as a single-page web app (section 17).
Dry run	Notification mode in which messages are recorded in the response but not sent.
ChannelCode	The partner that owns a record (SOHAR = Sohar Bank); sent on every request and saved on every proposal.
OrgCode	Issuing branch code; the platform validates it against the code table PubBranch (value 1).
Carrier / insurer	The insurer of a product: <code>CarrierCode</code> AFIC, name from table <code>UP_Carrier</code> .
Trace id	Platform request id, returned as <code>trace_id</code> and used to find the log of a call.

1.4 Read This First: What Will Surprise You

These are the facts most likely to cost a new developer a day. Each one is explained in the section named in the last column.

#	What	Why it matters	Section
1	The MC tenant runs with <code>RoleEnforcement = Y</code> , <code>AllowSelfApproval = Y</code> and <code>NotifyMode = DRYRUN</code> . <code>UP_UserRole</code> holds three test rows for one account (ravi.teja@insuremo.com holds all three roles).	One account can run the whole flow, so maker and checker are the same person. Before UAT or go-live: <code>AllowSelfApproval = N</code> , replace the test rows with bank staff.	15, 19
2	The maker's platform user id is stored in the proposal field AgentCode . Queues, "My submissions" and the maker-checker rule all depend on it.	<code>AgentCode</code> is a platform field that normally means a sales agent. It is the only searchable field available. It is in the server-owned list, so callers cannot set it. Proposals created before v1.2 have no maker id.	15.3
3	The work queues and dashboard counts are read from the platform search index , not from our tables. The index limit is 1,000 rows per query, it has no channel, no maker (except <code>AgentCode</code>), an empty insured name and a zero premium.	Every queue row is enriched by loading the proposal (one call per row). Page size x page number must not exceed 1,000. A date window is always applied.	15.4
4	Dashboard premium and commission are calculated by loading policies , capped at <code>DashboardMaxPolicies</code> (60) newest per product and flagged <code>Truncated</code> .	Totals are exact only while a product has no more policies than the cap in the window. The dashboard takes 7 to 10 seconds. It is not a reporting database.	15.5
5	The MC tenant holds a lot of test data : several hundred pending test proposals and over a hundred pending test cancellations created by the test suites.	The queues and dashboard show them. There is no API to delete them.	19
6	No email or SMS is delivered. The MC tenant has zero SNS email and SMS accounts. The SNS SMS request carries no free text, only a template code configured in the SNS account.	<code>NotifyMode = LIVE</code> has never been exercised. The code path compiles but is unproven (field names, template codes).	16
7	The portal's sign-in screen posts the user name and password to the platform's CAS ticket service and takes the token from the reply. Only the error path and the "use my InsureMO session" path were tested. A successful password sign-in was never run (no password available).	The reply field name is assumed (<code>access_token</code> / <code>token</code> / <code>ticket</code> , as in the Postman collection). Verify with a real login first.	17.3
8	The portal builds API URLs with the standard UIC helper (<code>/{{tenant_code}}/api-orchestration-test/v1/flow</code>). All automated tests ran with an override (<code>sohar_api_base</code> in <code>sessionStorage</code>) that points the page at the gateway through a stub. The URL construction inside the real portal frame was not exercised by a test.	If the first call in the real portal fails with 404, check the route prefix. The override hook should be removed before go-live.	17.4
9	The <code>imo login</code> (<code>imo auth login</code>) lasts 24 hours and cannot be refreshed from a script. Every tool in the build folder needs a valid profile token.	When a push or script says "token is invalid or expired", log in again; nothing is wrong with the code.	19
10	PA001 and its APIs are a different design : iComposer APIs in group <code>Policy_API</code> that call an adapter (<code>IhubCallerIntegrationAdapter</code>) driven by the table <code>AdapterServiceCfg</code> . The new /up APIs do not use that adapter or iHub.	Do not reuse PA001 helper functions or tables for the new products. PA001 must stay untouched (<code>snapshot.py</code>).	18

#	What	Why it matters	Section
11	Cancellation requests that are rejected are indistinguishable from requests replaced when an approval runs: both end as endorsement status 600 (withdrawn).	The cancellation queue can only list PENDING (120) and ISSUED (300).	15.4
12	The specification (v1.1), UAT plan, policy model, OpenAPI file and Postman collection were not updated for v1.2. This guide is the only document that describes the new APIs.	Section 15.7 holds request and response examples. Update the other documents before they are given to partners.	15.7, 19
13	GET /up/me omits the Roles list when the caller has no roles (the platform drops empty lists).	Treat a missing Roles as an empty list (the portal does).	15.2
14	An inactive probe API UPspikeWho (path /up/spike-who) remains in MC from the identity spike. The imo tool cannot delete an API.	Delete it in the iComposer console before UAT. It must not be deployed.	19
15	Static type checking on push: UPShaper.shapeResponse returns Object (cast it), SMS and email builders take requestBody(...), an anonymous Comparator may not use method variables.	Compile errors on push usually mean one of these; the fix is a cast or a different method name.	3.3

1.5 Release History

Version	What it added
1.0	13 APIs for 7 products (get plans, proposal, update, issue, load, reject, documents, cancel check and approve, master data, commission, feed file), table-driven validation, security controls, full test suite, collections.
1.1	Insurer (CarrierCode / CarrierName, table UP_Carrier) in every response and an optional validated request field; developer guide and UAT plan.
1.2	Roles (Maker, Proposal Checker, Cancellation Checker) with maker-checker rules; work queues, cancellation detail, dashboard, share by email / SMS and automatic alerts; the staff portal (UIC page) with sign-in; 5 new APIs, 3 new functions, 2 new tables, about 32 new configuration keys and 29 new field rules; analysis of PA001 and iHub.

2. Architecture

2.1 Layers

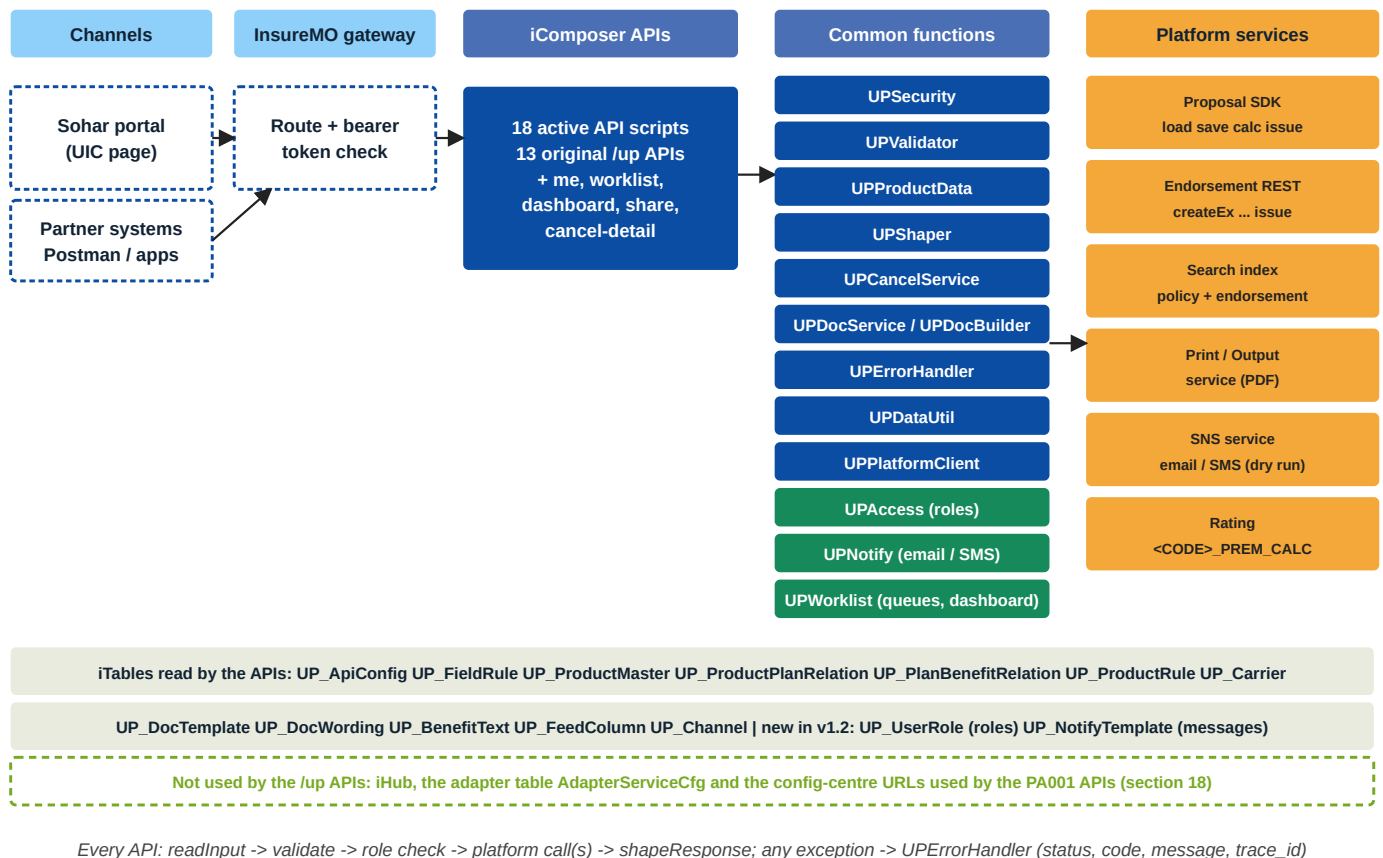


Figure 1: Architecture (v1.2) — channels, gateway, iComposer APIs, common functions, platform services and tables

Every API follows the same pipeline. The API script is thin: it reads the body through `UPSecurity.readInput`, validates it with `UPValidator.validate` (rules in table `UP_FieldRule` plus product and cross-field rules in code), calls the platform through the SDK clients (`ProposalSdkClient`) or the REST helper (`UPPlatformClient`), shapes the answer with `UPShaper.shapeResponse` and returns it. Any exception is turned into the standard error body by `UPErrorHandler.handle`, which also sets the HTTP status. The tables are read-only from the code (the `TablesSdkClient` is read-only); they are changed with the `imo` tool by an administrator. From v1.2 the API also checks the caller's role (`UPAccess`) and the platform search index is read for queues and the dashboard (`UPWorklist`).

2.2 Design Rules

- Everything configurable lives in tables: a missing configuration key stops the request with `UP-CONFIG` (HTTP 500, generic message, detail in the log); there is no silent default. The only exceptions are the security baselines in `UPSecurity`, which have an optional table override.
- PA001 (the original product, config and code) is never read or changed by this solution: product data lives in separate `UP_` tables and the new products are new platform products. A snapshot tool (`snapshot.py`) proves this before and after every change.
- The caller's own bearer token is used for every platform call; there is no service account. The token is only sent to hosts listed in `UP_ApiConfig.AllowedHosts` (https only).
- Responses are shaped for the channel: platform technical fields may be present but are not part of the contract.
- One error contract on every API: `status`, `code`, `message`, `errors[]`, `api`, `trace_id`. A missing role or a maker-checker breach is 403 `UP-403`.
- Roles come from the table `UP_UserRole` and are checked in the APIs, never in the portal alone. The portal menu is a convenience.

- No workflow state is kept in our own tables: proposals and cancellation requests are read from the platform (section 15).

2.3 Request Lifecycle

1. Gateway authenticates the bearer token (HTTP 401 on failure) and routes `/up/<path>` to the iComposer API.
2. API script: `UPSecurity.readInput(RequestBody())` — body must be a JSON object, then every string is scanned for blocked characters (400 `INVALID_BODY` / `INVALID_CHARACTER`).
3. `UPValidator.validate(api, input)` — rules from `UP_FieldRule` for this API (required, type, length, pattern, range, enum, code table, conditional `WhenPath`), then product rules (age, term, premium mode, plan availability), `CarrierCode` match, beneficiary total. All problems are returned together as 400 `UP-VALIDATION`.
4. Role check: `UPAccess.require(...)` (403 `UP-403`), then for approvals the maker-checker rule. State and ownership checks for existing records: record exists (404), belongs to the channel (`requireChannel`), product matches, proposal still open (`requireOpenProposal`, 409).
5. Platform call(s): `ProposalSdkClient` (application, load, save, `persistCalculate`, issue, `queryPolicy`) or `UPPlatformClient` (endorsement REST) or the print service.
6. `UPShaper.shapeResponse`: commission switch, premium fields, payment info restored, `CarrierCode` / `CarrierName` added. Then, where the action triggers one, `UPNotify.fire` adds `Notifications[]`.
7. Return the map (HTTP 200) or, on any exception, `UPErrorHandler.handle(apiName, ex)`.

3. Source Layout and Tooling

3.1 Workspace Layout

```
ic/
  src/dev/Tenant/UP_PRODUCTS_API/
    api/<Name>/<Name>.groovy      20 API scripts (18 active; UPCancelPolicy is a retired stub and UPSpikeWho
a probe, both inactive)
    function/<Name>/<Name>.groovy  13 common functions
    .metadata/icomposer/{api,function}/<Name>.metadata.json  per-profile server metadata (ids, version, md5,
path, method)
uponly-build/
  tables/                        build, test and report tooling (section 11)
                                the rows written to the UP_ tables (r_*.json), definitions (def_*.json),
local copies (rows_*.json)
  samples/                      recorded request / response of every flow step
  tests/ security/ carrier/ workflow/ test cases and results
uic_pages/uponly/portal/sohar-insurance-portal/  the portal (index.html, metadata.json, prompt.md) in the git
repository
deliverables/workflow_src/      v1.2 iComposer source, set-up and test scripts (git copy)
```

Module id 70, group id 1188491119; functions have scope `public` and status 1. APIs are POST except `UPLoadPolicy` and `UPMe` (GET). All paths are `/up/<name>`: `get-plans`, `proposal`, `update-proposal`, `issue`, `load-policy`, `proposal-reject`, `doc_generation`, `application_doc_generation`, `cancel-check`, `cancel-approve`, `list-master-table`, `commission-query`, `generate-feedfile`, and since v1.2 me, `worklist` (`UPWorklistApi`), `dashboard`, `share`, `cancel-detail` (the inactive ones are `cancel-policy` and `spike-who`).

3.2 The imo Tool

Task	Command (profile <code>portal:uponly</code> = MC)
Push one file (compiles on the server first)	<code>imo icomposer push current <file> --profile portal:uponly</code>
Preview a push	<code>... push current <file> --dry-run</code>
Create a function / API	<code>imo icomposer create function --name X --module-id 70 --group-id 1188491119 --status 1 --func-scope 1 --profile portal:uponly</code>
Pull the server copy	<code>imo icomposer reload ...</code> ; on a conflict choose <code>--prefer-local</code> or <code>--prefer-server</code>
Read a table / rows	<code>imo config datatable by-name --name UP_ApiConfig --json, ... record-list --id <tableId> --json</code>
Add rows	<code>imo config datatable record-batch-save --file rows.json --confirm</code> (insert only; the primary key <code>Id</code> must be unique)
Change one row	<code>imo config datatable record-save --file row.json --confirm</code> (include <code>RecordId</code>)
Create a table	<code>imo config datatable create --file def.json --confirm</code>
Find a call in the log	<code>imo log query</code> with the trace id from the response (section 9)

3.3 Groovy Rules of This Runtime

- The server type-checks Groovy statically on push. `@groovy.transform.Field` is not allowed and script-level variables are not visible inside methods: use a method that returns the constant (see `UPSecurity.blockedBaseline()`).
- `log` is not declared: use `org.slf4j.LoggerFactory.getLogger("Name")` (see `UPErrorHandler`).
- Shared per-request state goes into the request attributes (`RequestContextHolder`), as the table cache in `UPDataUtil.getRecords` does.
- A function that calls another one declares it with `(UPOther) getCommonService("UPOther")`; push the called function first when it is new.
- The platform REST client raises its own exception type; `UPPlatformClient` catches it and returns `{status, body}` so that callers decide.
- `UPShaper.shapeResponse` returns `Object`: cast it `((Map<String, Object>) shaper.shapeResponse(r))` before adding fields. The SNS email and SMS builders take `requestBody(...)`; an anonymous `Comparator` must

- not use method variables; the accessor of a hyphenated SDK tag is camel case (`emailAccountApi()`).
- Empty lists are dropped from the JSON that the platform returns (for example `Roles` in `/up/me`): treat a missing list as empty.
- Keep each function focused; the largest file is `UPValidator`.

3.4 Deployment Order of the Code

Dependencies run from the bottom up. Push in this order (the same order is used for UAT): `UPDataUtil`, `UPSecurity`, `UPErrorHandler`, `UPPlatformClient`, `UPValidator`, `UPProductData`, `UPShaper`, `UPDocBuilder`, `UPDocService`, `UPAccess`, `UPNotify`, `UPWorklist`, `UPCancelService`, then the 18 active APIs. The tables `UP_UserRole` and `UP_NotifyTemplate` and the new configuration keys must exist before the new code runs. Pushing a caller before a new callee fails the compile check.

4. Code Reference

4.1 Asset Inventory

Name	Kind	Route	Status	Lines	Server version	Calls (common functions)
UPApplicationDocGeneration	API	POST /up/application_doc_generation	active	25	8	UPAccess, UPDataUtil, UPDocService, UPErrorHandler, UPSecurity, UPValidator
UPCancelApprove	API	POST /up/cancel-approve	active	25	6	UPAccess, UPCancelService, UPDataUtil, UPErrorHandler, UPSecurity, UPValidator
UPCancelCheck	API	POST /up/cancel-check	active	26	6	UPAccess, UPCancelService, UPDataUtil, UPErrorHandler, UPSecurity, UPValidator
UPCancelDetail	API	POST /up/cancel-detail	active	28	2	UPAccess, UPCancelService, UPDataUtil, UPErrorHandler, UPSecurity, UPValidator
UPCancelPolicy	API	POST /up/cancel-policy	inactive	5	6	
UPCommissionQuery	API	POST /up/commission-query	active	137	11	UPAccess, UPDataUtil, UPDocBuilder, UPErrorHandler, UPProductData, UPSecurity, UPValidator
UPDashboard	API	POST /up/dashboard	active	50	2	UPAccess, UPDataUtil, UPErrorHandler, UPSecurity, UPValidator, UPWorklist
UPDocGeneration	API	POST /up/doc_generation	active	24	8	UPAccess, UPDataUtil, UPDocService, UPErrorHandler, UPSecurity, UPValidator
UPFeedFile	API	POST /up/generate-feedfile	active	178	9	UPDataUtil, UPErrorHandler, UPSecurity, UPShaper, UPValidator
UPGetPlans	API	POST /up/get-plans	active	105	8	UPAccess, UPDataUtil, UPErrorHandler, UPProductData, UPSecurity, UPShaper, UPValidator
UPIssue	API	POST /up/issue	active	58	10	UPAccess, UPDataUtil, UPErrorHandler, UPNotify, UPSecurity, UPShaper, UPValidator
UPListMasterTable	API	POST /up/list-master-table	active	37	7	UPDataUtil, UPErrorHandler, UPSecurity, UPValidator
UPLoadPolicy	API	GET /up/load-policy	active	37	8	UPAccess, UPDataUtil, UPErrorHandler, UPShaper, UPValidator
UPMe	API	GET /up/me	active	13	2	UPAccess, UPErrorHandler
UPProposal	API	POST /up/proposal	active	44	7	UPAccess, UPDataUtil, UPErrorHandler, UPNotify, UPProductData, UPSecurity, UPShaper, UPValidator
UPProposalReject	API	POST /up/proposal-reject	active	50	10	UPAccess, UPDataUtil, UPErrorHandler, UPNotify, UPSecurity, UPValidator
UPShare	API	POST /up/share	active	72	2	UPAccess, UPDataUtil, UPErrorHandler, UPNotify, UPProductData, UPSecurity, UPValidator, UPWorklist
UPSpikeWho	API	POST /up/spike-who	inactive	20	6	

Name	Kind	Route	Status	Lines	Server version	Calls (common functions)
UPUpdateProposal	API	POST /up/update-proposal	active	62	10	UPAccess, UPDataUtil, UPErrorHandler, UPProductData, UPSecurity, UPShaper, UPValidator
UPWorklistApi	API	POST /up/worklist	active	35	2	UPAccess, UPDataUtil, UPErrorHandler, UPSecurity, UPValidator, UPWorklist
UPAccess	Function	-	active	124	4	UPDataUtil
UPCancelService	Function	-	active	364	8	UPAccess, UPDataUtil, UPErrorHandler, UPNotify, UPPlatformClient, UPProductData, UPValidator, UPWorklist
UPDataUtil	Function	-	active	235	9	
UPDocBuilder	Function	-	active	195	3	UPDataUtil, UPProductData
UPDocService	Function	-	active	105	4	UPDataUtil, UPDocBuilder, UPErrorHandler, UPShaper, UPValidator
UPErrorHandler	Function	-	active	104	10	UPDataUtil
UPNotify	Function	-	active	166	4	UPAccess, UPDataUtil, UPProductData
UPPlatformClient	Function	-	active	49	7	UPDataUtil
UPProductData	Function	-	active	213	5	UPDataUtil, UPSecurity, UPShaper
UPSecurity	Function	-	active	116	5	UPDataUtil
UPShaper	Function	-	active	126	3	UPDataUtil, UPProductData
UPValidator	Function	-	active	334	3	UPDataUtil, UPProductData
UPWorklist	Function	-	active	408	4	UPAccess, UPDataUtil, UPPlatformClient, UPProductData, UPValidator

Total: 3570 lines of Groovy (1031 in APIs, 2539 in functions).

4.2 Common Functions

UPAccess (124 lines)

UPAccess - who is calling and what may they do (Maker / Proposal Checker / Cancellation Checker). The caller is taken from the platform token (IComposerAppContext), never from the request. Roles come from the UP_UserRole table. Enforcement is switched by UP_ApiConfig RoleEnforcement (TrueValue = on); with it off the APIs behave as before.

tables read: UP_UserRole | config keys used: AllowSelfApproval, MsgForbidden, MsgNotOwner, MsgSelfApproval, RoleCancellationChecker, RoleEnforcement, RoleMaker, RoleProposalChecker | calls: UPDataUtil

Method	Parameters	Returns	Purpose
currentUser		Map<String, Object>	The platform user behind the token: UserName, UserId, DisplayName, Email.

Method	Parameters	Returns	Purpose
roleRows	String userName, String userId	List<Map<String, Object>>	Active role rows of a user (by user name, case-insensitive, or by platform user id).
me		Map<String, Object>	Caller identity with roles, branch and contact details. Roles is empty when the user is not in UP_UserRole.
require	List<String> allowedRoles	Map<String, Object>	Stops the call (HTTP 403, UP-403) unless the caller holds one of the roles. Returns the caller identity. With RoleEnforcement off nothing is checked.
requireNotCreator	Map<String, Object> caller, String creatorUserId	void	Maker-checker rule: the person who created an item cannot approve or reject it (unless AllowSelfApproval is on).
requireAny		Map<String, Object>	Any role holder (read access to policy data).
requireCreator	Map<String, Object> caller, String creatorUserId	void	A maker may only change what they created.
holdersOf	String role	List<Map<String, Object>>	Role holders (for staff alerts): rows with the role that have an email or mobile number.
contactOf	String userId	Map<String, Object>	One user's contact details by platform user id (the maker of a proposal); empty map when unknown.

UPCancelService (364 lines)

UPCancelService - cancellation in two steps (Checker, then Approver). * The request itself is the platform's own pending cancellation endorsement: the Checker creates it (the endorsement number is the RequestNo), the Approver approves it (calculate, validate, issue) or rejects it (suspend). No premium is calculated by the platform here; the refund is calculated by UPDataUtil.refundCalc from the product rules. Every code (statuses, types, paths) comes from UP_ApiConfig.

tables read: ProductMaster | SDK: ProposalSdkClient | config keys used: AllowSelfApproval, CancellInstallmentFlag, CancellInstallmentType, CancelPayMode, CancelReasonCode, CancelTypeFreeLook, CancelTypeProRata, DecisionApprove, EndoBasePath, EndoTypeCancel, PlatformErrorMaxLength, PolicyStatusInforce, ProposalLoadPath, RefundTypeFreeLook, RefundTypeProRata, RequestStatusApproved, RequestStatusPending, RequestStatusRejected, RoleEnforcement | calls: UPAccess, UPDataUtil, UPErrorHandler, UPNotify, UPPlatformClient, UPProductData, UPValidator, UPWorklist

Method	Parameters	Returns	Purpose
resolvePolicy	Map<String, Object> input	Map<String, Object>	The issued policy named by PolicyNo or ProposalNo, checked against the caller's channel. Returns null when it does not exist.
pendingCancellation	Map<String, Object> policy	Map<String, Object>	The pending cancellation endorsement of the policy, or null. Another kind of pending endorsement blocks the request.
cancellationDate	Map<String, Object> endorsement	LocalDate	Cancellation date used for the refund: the endorsement effective date, or the start of the policy for a free-look cancellation.
policyInfo	Map<String, Object> policy	Map<String, Object>	Policy information block shared by both responses.
policyStatus	Map<String, Object> policy	Map<String, Object>	

Method	Parameters	Returns	Purpose
createRequest	Map<String, Object> policy, String effectiveDate, boolean freeLook, String reason	Map<String, Object>	Creates the pending cancellation endorsement (the request). No premium is calculated by the platform here.
suspend	Map<String, Object> endorsement	void	Withdraws a pending endorsement (reject, or before it is replaced on approval).
check	Map<String, Object> input	Map<String, Object>	Checker: creates (or returns) the pending cancellation request and shows premium and refund details.
detail	Map<String, Object> input	Map<String, Object>	Checker view of a pending request (nothing is created): who asked, why, and the refund an approval would pay.
decide	Map<String, Object> input	Map<String, Object>	Approver: approves (cancels the policy through the platform) or rejects the pending request named by RequestNo.
refundCalc	Map<String, Object> policy, java.time.LocalDate cancelDate	Map<String, Object>	Refund rule for a cancellation: full premium within the free-look period (counted from the commencement date), otherwise the percentage configured in UP_ProductRule (ProRataRefundPercent) of the pro-rata premium. Premium = plan premium on the risk (before tax). FreeLookDays and ProRataRefundPercent come from ProductRule.

UPDataUtil (235 lines)

UPDataUtil - core helpers: configuration (UP_ApiConfig), table access, flag values, formats, error helpers.

tables read: UP_ApiConfig | SDK: TablesSdkClient | config keys used: AllowedHosts, DateFormatDoc, FalseValue, MoneyFormat, MoneyScale, NumberFormat, PercentBase, RoundingMode, TrueValue

Method	Parameters	Returns	Purpose
fail	String message	void	
str	Object value	String	
yes		String	Flag values come from UP_ApiConfig (TrueValue / FalseValue), never from literals in code.
no		String	
isYes	Object value	boolean	
isNo	Object value	boolean	
requireAllowedHost	String url	void	The URL's host must be listed in UP_ApiConfig AllowedHosts and the scheme must be https before a bearer token is sent to it.
fill	String text, Map<String, String> values	String	Replaces {name} markers in a configured text.
roundingMode		java.math.RoundingMode	
moneyScale		int	
percentBase		BigDecimal	
configMap	String scope	Map<String, String>	Configuration value from UP_ApiConfig (scope = product code or ""). A missing key is a configuration error, never a silent default.

Method	Parameters	Returns	Purpose
cfgValue	Map<String, String> config, String key	String	
cfg	String key, String scope	String	
pathValue	Object root, String path	Object	Value at a path such as PolicyLobList[0].PolicyRiskList[0].DateOfBirth.
fieldError	String field, String code, String message	Map<String, Object>	
throwValidation	List<Map<String, Object>> errors	void	
parseDate	Object value	LocalDate	
getRecords	String tableName	List<Map<String, Object>>	All records of a datatable as plain maps. Read once per request (kept in the request attributes), so data changes are visible on the next request.
readRecords	String tableName	List<Map<String, Object>>	
filterRecords	String tableName, Map<String, Object> filters	List<Map<String, Object>>	
toDecimal	Object value	BigDecimal	
fmtDate	Object value	String	Display date, pattern from UP_ApiConfig DateFormatDoc.
fmtMoney	Object value	String	Display amount, pattern from UP_ApiConfig MoneyFormat.
fmtNumber	String v	String	Display number, pattern from UP_ApiConfig NumberFormat.

UPDocBuilder (195 lines)

UPDocBuilder - print data for the application form / certificate, document wording, commission view.

tables read: UP_BenefitText, UP_DocWording | config keys used: PlanLabelPrefix | calls: UPDataUtil, UPProductData

Method	Parameters	Returns	Purpose
describe	String tableName, Object id	String	
benefitText	Map<String, Object> benefit	String	Benefit limit wording from UP_DocWording (ModeCode = BASIS: <LimitBasis>, falling back to the BASIS:Default row of the same table).
docBenefit	String productCode, String code	List<String>	Official English / Arabic benefit names (UP_BenefitText): [english, arabic]. A row is required for every benefit that is printed.
premiumWording	String modeCode, String term, String premium	Map<String, Object>	English / Arabic wording of the premium frequency, premium text and period of cover for a premium mode (UP_DocWording).
buildPrintData	Map<String, Object> policy, String docType	Map<String, Object>	
rate	Object commission, Object premium	String	
commissionOfPolicy	Map<String, Object> policy	Map<String, Object>	Commission view of one loaded policy (commission is read from the stored policy, not from configuration).

UPDocService (105 lines)

UPDocService - builds the application form or the policy certificate as a PDF through the print service. Shared by UPDocGeneration and UPApplicationDocGeneration. Every setting (template version, print URL, export type, storage, mode, file name) comes from UP_ApiConfig, the template name from UP_DocTemplate.

tables read: DocTemplate | SDK: ProposalSdkClient | config keys used: DocContentType, DocFileNamePattern, DocTypeApplication, DocTypePolicy, PrintExportType, PrintMode, PrintStorage, PrintUrl, TemplateVersion | calls: UPDataUtil, UPDocBuilder, UPErrorHandler, UPShaper, UPValidator

Method	Parameters	Returns	Purpose
generate	String apiName, String docType, Map<String, Object> input	Object	Returns the PDF (HttpEntity), the print data when PreviewData is the TrueValue, or a 404 body.

UPErrorHandler (104 lines)

UPErrorHandler - turns an exception into a structured error response and sets the HTTP status. Validation problems return 400, missing role or maker-checker violations 403, state conflicts 409, platform API errors keep their status; configuration problems and anything else are 500 with a generic table-driven message (the detail goes to the log, the caller gets the trace id).

config keys used: MsgConfigError, MsgInvalidNumber, MsgPlatformInput, MsgServerError | calls: UPDataUtil

Method	Parameters	Returns	Purpose
configText	String key, String technical	String	Message text from UP_ApiConfig; the technical fallback is used only when the configuration itself cannot be read.
isInternalText	String message	boolean	True when a platform message carries database or framework internals (index names, class names, parser positions) that must not reach the caller.
handle	String apiName, Exception ex	Map<String, Object>	
notFound	String apiName, String message	Map<String, Object>	

UPNotify (166 lines)

UPNotify - Email / SMS through the tenant's SNS service, driven by UP_NotifyTemplate. A notification never fails the business call: every attempt returns a result row {Event, Recipient, Channel, To (masked), Status}. Status: SENT, DRYRUN (NotifyMode is not LIVE), NO_ADDRESS (no email / mobile known), NO_SMS_TEMPLATE (SmsTemplateCode is NONE) or FAILED. Email is sent with subject and body from the template. The SNS SMS request carries no free text: it names an SMS template (SmsTemplateCode) of the SNS account and its parameters.

tables read: UP_NotifyTemplate | SDK: SnsSdkClient | config keys used: EmailAccountName, NotifyMode, PortalUrl, SmsAccountName | calls: UPAccess, UPDataUtil, UPProductData

Method	Parameters	Returns	Purpose
maskEmail	String email	String	
maskMobile	String mobile	String	
render	String text, Map<String, String> params	String	Template text with {Name} markers replaced; markers without a value become empty.
resultRow	String event, String recipient, String channel, String to, String status	Map<String, Object>	
sendOne	String channel, String to, String subject, String body, String smsTemplate, Map<String, String> params	String	

Method	Parameters	Returns	Purpose
fire	String event, Map<String, String> params, Map<String, Object> customer, String makerUserId, String onlyChannel	List<Map<String, Object>>	Sends every active template of the event. Recipients: a role (all active holders in UP_UserRole), MAKER (the creator of the item, by platform user id) or CUSTOMER (customer map with Email / Mobile). onlyChannels restricts to EMAIL / SMS (empty = both).
paramsOf	Map<String, Object> policy, Map<String, String> extra	Map<String, String>	Placeholder values for the templates from a policy / proposal map (stored or shaped). extra overrides / adds values.
customerOf	Map<String, Object> storedPolicy	Map<String, Object>	Customer contact (Email, Mobile) of a stored proposal / policy, unmasked, for customer alerts.

UPPlatformClient (49 lines)

UPPlatformClient - calls platform services (endorsement) with the caller's own token. Returns { status: <http status>, body: <response text> } and never throws for HTTP or platform errors.

config keys used: GatewayUrl | calls: UPDataUtil

Method	Parameters	Returns	Purpose
call	String method, String path, Object body	Map<String, Object>	
parse	Map<String, Object> response	Map<String, Object>	

UPProductData (213 lines)

UPProductData - product master data (variants, rules, plans, benefits) and preparation of the proposal payload.

tables read: ProductMaster, ProductPlanRelation, ProductRule, UP_Carrier, UP_PlanBenefitRelation | config keys used: RiskElementCode | calls: UPDataUtil, UPSecurity, UPShaper

Method	Parameters	Returns	Purpose
getVariant	String productCode, String subcode	Map<String, Object>	
carrierOf	String productCode, String subcode	Map<String, Object>	Insurer (carrier) of a product variant: the code is on the product (UP_ProductMaster.CarrierCode), the name comes from UP_Carrier. A product without a carrier, or a carrier that is missing or inactive in UP_Carrier, is a configuration error (UP-CONFIG), never a default.
addCarrier	Map<String, Object> target, String productCode, String subcode	void	Adds CarrierCode and CarrierName (the insurer of the product) to a response map.
getRule	String productCode, String subcode	Map<String, Object>	
allowedModes	Map<String, Object> rule	List<String>	
getPlanRows	String productCode, String subcode, String premiumModeCode	List<Map<String, Object>>	Plan rows (premium + commission) for the variant and premium mode.
getBenefitRows	String productCode, String subcode, String planCode	List<Map<String, Object>>	
buildPlanView	String productCode, String subcode, String premiumModeCode	List<Map<String, Object>>	Plans for the front end: premium and benefits only, never commission.

Method	Parameters	Returns	Purpose
prepareProposal	Map<String, Object> input	Map<String, Object>	Validates the request against product data and fills the authoritative values (plan premium, commission, coverages when none are supplied). Returns the payload ready for the native proposal SDK.
variantName	String productCode, String subcode	String	Display name of a product variant (UP_ProductMaster CarrierProductName); empty when unknown.

UPSecurity (116 lines)

UPSecurity - request hygiene shared by every API. readInput the body must be a JSON object (anything else is a 400, never a 500); it is scanned before any API logic runs scan blocked characters in every text value, extra allow-list for free-text fields (400 INVALID_CHARACTER) stripServerOwned fields the server owns (policy number, ids, status) are never taken from the caller defaultOrgCode a proposal always carries a branch that the platform code table PubBranch accepts The baselines below are security controls: when UP_ApiConfig has the key (BlockedCharacters, FreeTextFields, FreeTextPattern, ServerOwnedFields, OrgCode) the table value is used, otherwise the built-in baseline applies (never an error).

calls: UPDataUtil

Method	Parameters	Returns	Purpose
blockedBaseline		String	Control characters and the characters used for script, template, shell and expression injection.
freeTextBaseline		String	Free-text fields: letters of any script, digits, space and . , ' - / @ + () only.
freeTextFieldsBaseline		String	
serverOwnedBaseline		String	
orgCodeBaseline		String	
maxFindings		int	
setting	String key, String baseline	String	Optional table override of a baseline.
readInput	Object body	Map<String, Object>	The request body as a map, already checked by scan(). Null is an empty request; an array, text or number is a 400.
scan	Map<String, Object> input	void	Rejects blocked characters in any text value, and anything outside the allow-list in the free-text fields. The value is never echoed.
scanValue	Object value, String path, String key, Pattern blocked, Pattern freeText, Set<String> freeFields, List<Map<String, Object>> errors, UPDataUtil util	void	
stripServerOwned	Map<String, Object> input	void	Removes the top-level fields the server owns. The platform sets the proposal status itself; a status sent by the caller made the platform fail (HTTP 500 / 502), so it is dropped.
defaultOrgCode	Map<String, Object> input	void	A branch the platform accepts (code table PubBranch); the caller's own OrgCode is kept.

UPShaper (126 lines)

UPShaper - shapes the responses returned to callers (commission, premium fields, payment info).

config keys used: InstalmentAmount, InstalmentFeeSeq, InstalmentPayRate, InstalmentPeriodCount, InstalmentPeriodSeq, PolicyStatusEffective, ReturnCommission, SequenceNumberStart, ZeroPremiumFields, ZeroPremiumValue | calls: UPDataUtil, UPProductData

Method	Parameters	Returns	Purpose
stripCommission	Object node	Object	Removes commission from a policy / proposal body before it is returned to a front-end caller.
restorePayment	Object body	Object	Rebuilds PolicyPaymentInfoList from the flattened payment fields.
returnCommission		boolean	Commission is returned only while UP_ApiConfig ReturnCommission equals the TrueValue.
enrichPremiumFields	Map<String, Object> map	void	Adds the premium structure fields the PA001 responses carry (zero where the level is not rated) and the plan row premium / commission.
shapeResponse	Object body	Object	Response shaping for every front-end facing API: same structure as the PA001 flow, payment info in its original shape.

UPValidator (334 lines)

UPValidator - request validation: field rules (UP_FieldRule), cross-field and product rules, state / channel checks.

tables read: ProductMaster, UP_FieldRule | config keys used: BeneficiaryShareTotal, EffectiveDateMinOffsetDays, MasterTables, NonSingleTermYears, PatternMaxInput, PatternMaxLength, ProposalStatusIssued, ProposalStatusRejected, SinglePremiumModeCode | calls: UPDataUtil, UPProductData

Method	Parameters	Returns	Purpose
requireOpenProposal	Map<String, Object> stored, String action	void	A proposal can only be updated / issued / rejected while it is open (status codes come from UP_ApiConfig).
requireProductMatch	Map<String, Object> stored, String productCode, String subcode	void	The stored proposal / policy must belong to the product the caller names.
requireChannel	Map<String, Object> stored, String channelCode	void	The stored proposal / policy must have been created by the channel that calls (ChannelCode is saved on every proposal).
codeExists	String table, String column, String value	boolean	
validate	String api, Map<String, Object> input	void	Field level validation. Rules come from UP_FieldRule (per API) and product rules from UP_ProductRule. All problems are collected and returned together as HTTP 400 with errors[].
crossFieldErrors	String api, Map<String, Object> input, boolean productFieldBad	List<Map<String, Object>>	Rules that need several fields or product data. Only runs on values that already passed the field rules.
entryErrors	Map<String, Object> rule, String dateOfBirth, String effectiveDate, String premiumModeCode, String policyTerm	List<Map<String, Object>>	Age, term and premium mode checks from UP_ProductRule, returned as field errors.

UPWorklist (408 lines)

UPWorklist - queues and dashboard figures read from the platform's search index (nothing is stored by this API). Proposal queue : proposals by ProposalStatus (pending / issued / rejected), per UP product. Cancellation queue : cancellation endorsements by EndoStatus (pending / issued). Maker filter : the maker's platform user id is stamped on every proposal in AgentCode by UPProductData.prepareProposal, so "my submissions" is an index

search. The index is limited to 1000 rows per query (page size x page number) and holds no premium, so premium and commission are read from the stored policies (capped by DashboardMaxPolicies).

tables read: ProductMaster | SDK: ProposalSdkClient | config keys used: DashboardMaxPolicies, DayEndTime, EndoSearchModule, EndoSearchPath, EndoStatusIssued, EndoStatusPending, EndoTypeCancelFilter, PolicyStatusInforce, ProposalStatusIssued, ProposalStatusPending, ProposalStatusRejected, SearchModule, SearchSortField, SearchSortType, WorklistMaxRows, WorklistPageSize | calls: UPAccess, UPDataUtil, UPPlatformClient, UPProductData, UPValidator

Method	Parameters	Returns	Purpose
productCodes		List<String>	Distinct product codes enabled for the APIs (UP_ProductMaster).
proposalStatusCode	String name	String	Status text for a status filter name, as configured. Returns "" for ALL.
dateRange	Map<String, Object> input, String key, boolean from	Map<String, Object>	
condition	Map<String, Object> conditions, Map<String, Object> from, Map<String, Object> to, int pageNo, int pageSize, String module	SearchCondition	
policySearch	Map<String, Object> conditions, Map<String, Object> from, Map<String, Object> to, int pageNo, int pageSize	Map<String, Object>	One policy-index search: {Total, Docs}.
endoSearch	Map<String, Object> conditions, Map<String, Object> from, Map<String, Object> to, int pageNo, int pageSize	Map<String, Object>	One endorsement-index search: {Total, Docs}.
loadPolicy	String proposalNo, String policyNo	Map<String, Object>	
makerName	String userId	String	Display name of a platform user id (UP_UserRole), else the id itself.
policyRow	Map<String, Object> policy	Map<String, Object>	Row shown in a queue, from the stored policy / proposal.
intOf	Object value, int fallback	int	
proposalList	Map<String, Object> input, String mineUserId	Map<String, Object>	Proposal queue: type PROPOSAL (all makers) or MINE (mineUserId is the caller's id), filtered by Status, product and dates.
cancellationList	Map<String, Object> input	Map<String, Object>	Cancellation queue: pending (default) or issued cancellation endorsements.
summary	Map<String, Object> input, String mineUserId	Map<String, Object>	Counts of proposals and cancellations in the date window (mineUserId limits proposals to one maker).
commissionSection	Map<String, Object> input, String productCode, String mineUserId	Map<String, Object>	Premium and commission of the issued policies of one product in the window, by plan. Capped at DashboardMaxPolicies newest policies (Truncated tells when the cap applied).

4.3 APIs

UPApplicationDocGeneration — POST /up/application_doc_generation (25 lines)

Generate the application form of a proposal as a PDF (same request as the PA001 flow). Request: { "proposalNo": "...", "ProductSubcode": "1" } The template is looked up in UP_DocTemplate by ProductCode + ProductSubcode + DocType.

calls: UPAccess, UPDataUtil, UPDocService, UPErrorHandler, UPSecurity, UPValidator | config keys: DocTypeApplication

UPCancelApprove — POST /up/cancel-approve (25 lines)

Cancellation - Approver. Approves (cancels the policy) or rejects the pending request created by the Checker. Request: { "ChannelCode": "SOHAR", "PolicyNo" | "ProposalNo": "...", "RequestNo": "<from the Checker>", "Decision": "APPROVE" | "REJECT" } Response: RequestNo, RequestStatus, PolicyInfo, PolicyStatus, RefundPremium.

calls: UPAccess, UPCancelService, UPDataUtil, UPErrorHandler, UPSecurity, UPValidator | config keys: RoleCancellationChecker

UPCancelCheck — POST /up/cancel-check (26 lines)

Cancellation - Checker. Creates the pending cancellation request for an issued policy and shows what an approval would do. Request: { "ChannelCode": "SOHAR", "PolicyNo" | "ProposalNo": "...", "CancellationDate": "2026-10-20", "CancelReason": "..." } Response: RequestNo, RequestStatus, PolicyInfo, PolicyStatus (current), PremiumDetails, RefundDetails. Repeating the call for a policy that already has a pending request returns that request (RequestExists = TrueValue).

calls: UPAccess, UPCancelService, UPDataUtil, UPErrorHandler, UPSecurity, UPValidator | config keys: RoleMaker

UPCancelDetail — POST /up/cancel-detail (28 lines)

Cancellation - details of a pending request for the Cancellation Checker (and the maker who raised it). Creates and changes nothing. Request: { "ChannelCode": "SOHAR", "PolicyNo" | "ProposalNo": "..." } Response: RequestNo, RequestedBy, RequestedOn, CancelReason, CancellationDate, PolicyInfo, PremiumDetails, RefundDetails, CanDecide.

calls: UPAccess, UPCancelService, UPDataUtil, UPErrorHandler, UPSecurity, UPValidator | config keys: RoleCancellationChecker, RoleMaker

UPCancelPolicy — POST /up/cancel-policy (5 lines)

Retired (inactive). Cancellation is now done in two steps: UPCancelCheck (/up/cancel-check) and UPCancelApprove (/up/cancel-approve).

UPCommissionQuery — POST /up/commission-query (137 lines)

Commission query - the only API that returns commission (all other APIs hide it). * 1. By policy: { "PolicyNo": "POFP00100000013" } 2. By configuration: { "ProductCode": "FP001", "ProductSubcode": "1", "PlanId": "2" (optional) } 3. By period: { "ProductCode": "FP001" (optional), "FromDate": "2026-10-01", "ToDate": "2026-10-31", "PageNo": 1, "PageSize": 50 } * Commission is configured per product / variant / plan in the ProductPlanRelation table (column Commission). It is written onto the policy when the premium is calculated, so a policy keeps the commission it was issued with.

calls: UPAccess, UPDataUtil, UPDocBuilder, UPErrorHandler, UPProductData, UPSecurity, UPValidator | SDK: ProposalSdkClient | tables: ProductPlanRelation | config keys: CommissionDefaultPageNo, CommissionDefaultPageSize, DayEndTime, DayStartTime, ProposalStatusIssued, SearchModule, SearchSortField, SearchSortType

UPDashboard — POST /up/dashboard (50 lines)

Dashboard figures. Two sections so the page can load them in parallel: SUMMARY counts of proposals (submitted / pending / issued / rejected, pending older than a day) and cancellations, per product. COMMISSION premium and commission of the issued policies of ONE product (ProductCode) in the window, by plan; cancelled policies are shown apart. Request: { "ChannelCode", "Section", "ProductCode" (COMMISSION), "Scope": "ALL" | "MINE", "FromDate", "ToDate" } (default window: the last 30 days, at most DashboardMaxDays). A caller who is only a Maker always sees their own submissions (Scope MINE).

calls: UPAccess, UPDataUtil, UPErrorHandler, UPSecurity, UPValidator, UPWorklist | config keys: DashboardMaxDays, RoleMaker

UPDocGeneration — POST /up/doc_generation (24 lines)

Generate the application form or the policy certificate as a PDF. Request: { "DocType": "Application" | "Policy", "ProposalNo": "...", "PolicyNo": "..." }

calls: UPAccess, UPDataUtil, UPDocService, UPErrorHandler, UPSecurity, UPValidator

UPFeedFile — POST /up/generate-feedfile (178 lines)

Generate the policy feed file (Excel by default, JSON when OutputFormat = "JSON"). Request: { "Conditions": {"ProductCode": "FP001"}, "FromRangeConditions": {"EffectiveDate": "..."}, "ToRangeConditions": {"EffectiveDate": "..."}, "SearchType": "Policy", "PageNo": 1, "PageSize": 50, "OutputFormat": "XLSX" | "JSON" }
Commission is not part of the feed file.

calls: UPDataUtil, UPErrorHandler, UPSecurity, UPShaper, UPValidator | SDK: ProposalSdkClient | tables: UP_FeedColumn | config keys: FeedContentType, FeedFileName, FeedMaxPages, FeedMaxRecords, FeedSheetName, ProposalStatusEntry, ProposalStatusIssued, SearchModule, SearchSortField, SearchSortType

UPGetPlans — POST /up/get-plans (105 lines)

Get plans for a product variant. Same shape as the PA001 flow: the request is the policy body (ProductCode + ProductSubcode, insured details), the response is that policy with every plan of the variant in PolicyRiskList[].PlanList (PlanId, PlanName, DuePremium, Commission, PolicyCoverageList). Optional: PremiumModeCode, PolicyTerm. Age and term rules are checked when the date of birth is supplied.

calls: UPAccess, UPDataUtil, UPErrorHandler, UPProductData, UPSecurity, UPShaper, UPValidator | config keys: BusinessCateCode, CustomerFlagNo, IsAutoAttachedForm, IsPremiumCalcSuccess, PolicyStatusEffective, RiskElementCode, SequenceNumberStart, ZeroPremiumFields, ZeroPremiumValue

UPIssue — POST /up/issue (58 lines)

Issue a policy from a proposal. Request: { "ProposalNo": "...", "ProductCode": "..." }.

calls: UPAccess, UPDataUtil, UPErrorHandler, UPNotify, UPSecurity, UPShaper, UPValidator | SDK: ProposalSdkClient | config keys: RoleProposalChecker

UPListMasterTable — POST /up/list-master-table (37 lines)

List master data for the front-end dropdowns. Request: { "DataTableName": "Relationship", "Id": "2" (optional) }.

calls: UPDataUtil, UPErrorHandler, UPSecurity, UPValidator | config keys: MasterTables

UPLoadPolicy — GET /up/load-policy (37 lines)

Load a policy (policyNo) or a proposal (proposalNo). Commission is removed from the response.

calls: UPAccess, UPDataUtil, UPErrorHandler, UPShaper, UPValidator | SDK: ProposalSdkClient

UPMe — GET /up/me (13 lines)

Who am I: the caller taken from the platform token with the roles configured in UP_UserRole. Response: UserName, UserId, DisplayName, Email, Branch, Roles[] (MAKER, PROPOSAL_CHECKER, CANCELLATION_CHECKER), RoleEnforcement, Insurer.

calls: UPAccess, UPErrorHandler

UPProposal — POST /up/proposal (44 lines)

Create a proposal (application). The request uses the native policy JSON (PolicyLobList / PolicyRiskList / PlanList / PolicyCoverageList / BeneficiaryList ...). The plan premium is taken from product data; coverages are filled from the plan when not supplied.

calls: UPAccess, UPDataUtil, UPErrorHandler, UPNotify, UPProductData, UPSecurity, UPShaper, UPValidator | SDK: ProposalSdkClient | config keys: RoleMaker

UPProposalReject — POST /up/proposal-reject (50 lines)

Reject a proposal (checker action). Request: { "ProposalNo", "ProposalRejectDesc" }.

calls: UPAccess, UPDataUtil, UPErrorHandler, UPNotify, UPSecurity, UPValidator | SDK: ProposalSdkClient | config keys: RoleProposalChecker, SuccessStatusText

UPShare — POST /up/share (72 lines)

Share a quotation, proposal or policy with the customer by Email and / or SMS (InsureMO SNS; dry run until the tenant has an account, see NotifyMode). Request: { "ChannelCode", "DocType": "QUOTATION" | "PROPOSAL" | "POLICY", "Channel": "EMAIL" | "SMS" | "BOTH", "Email", "Mobile", "ProposalNo" | "PolicyNo" (PROPOSAL / POLICY), "Details": { InsuredName, ProductName, PlanName, Premium, EffectiveDate } (QUOTATION) } Without Email / Mobile the customer details stored on the proposal / policy are used. Response: DocType, Channel, Notifications[] ({Event, Recipient, Channel, To (masked), Status}).

calls: UPAccess, UPDataUtil, UPErrorHandler, UPNotify, UPProductData, UPSecurity, UPValidator, UPWorklist | config keys: ShareDocQuotation

UPSpikeWho — POST /up/spike-who (20 lines)

Inactive probe API from the identity and SNS spike; delete it before UAT (section 19.4).

UPUpdateProposal — POST /up/update-proposal (62 lines)

Update an existing proposal. ProposalNo is required; the policy identifiers are loaded from the stored proposal.

calls: UPAccess, UPDataUtil, UPErrorHandler, UPProductData, UPSecurity, UPShaper, UPValidator | SDK: ProposalSdkClient | config keys: RoleMaker

UPWorklistApi — POST /up/worklist (35 lines)

Work queues for bank staff, read from the platform search index. Request: { "ChannelCode": "SOHAR", "Type": "PROPOSAL" | "CANCELLATION" | "MINE", "Status": "PENDING" | "ISSUED" | "REJECTED" | "ALL" (cancellations: PENDING | ISSUED), "ProductCode", "FromDate", "ToDate", "PageNo", "PageSize" } PROPOSAL = the Proposal Checker's queue, CANCELLATION = the Cancellation Checker's queue (Maker may read it), MINE = the caller's own submissions (Maker). Response: Type, Status, PageNo, PageSize, Total, Records[].

calls: UPAccess, UPDataUtil, UPErrorHandler, UPSecurity, UPValidator, UPWorklist | config keys: RoleCancellationChecker, RoleMaker, RoleProposalChecker

5. API Walkthroughs

What each API does in code, in the order it does it. Names in backticks are functions or methods listed in section 4.

API	Flow in code
v1.2 changes to existing APIs	UPProposal: UPAccess.require(Maker), prepareProposal, then AgentCode = caller id, create, shape, UPNotify.fire("SUBMITTED"), Notifications[] added. UPUdateProposal: Maker and creator only (requireCreator), keeps the stored AgentCode. UPIssue: Proposal Checker, requireNotCreator(existing.AgentCode), after the issue PROPOSAL_ISSUED to the maker and the customer (contact from the raw loaded proposal). UPProposalReject: Proposal Checker, not the creator, ProposalRejectDesc is required by a field rule, PROPOSAL_REJECTED to the maker. UPCancelCheck: Maker; a new request sends CANCEL_REQUESTED. UPCancelApprove: Cancellation Checker; UPCancelService.decide compares the requester (DataEntryUserId of the pending endorsement) with the caller and sends CANCEL_APPROVED / CANCEL_REJECTED. UPGetPlans, UPLoadPolicy, UPDocGeneration, UPApplicationDocGeneration and UPCommissionQuery call requireAny().
UPMe	UPAccess.me(); no validation, no role needed.
UPWorklistApi	Validates (UP_FieldRule rows for api name UPWorklist), role by Type, then UPWorklist.proposalList (PROPOSAL, MINE) or cancellationList. Section 15.4.
UPDashboard	Validates, requireAny(), forces own figures for a maker-only caller, checks the window, then UPWorklist.summary or commissionSection. Section 15.5.
UPShare	Validates, requireAny(), builds the placeholders (quotation: from Details; proposal / policy: loaded and channel-checked), takes the contact from the request or the stored record and calls UPNotify.fire(event, ..., onlyChannel). Section 16.
UPCancelDetail	Validates, Maker or Cancellation Checker, UPCancelService.detail: resolves the policy, finds the pending endorsement (signed policy id), recalculates the refund, returns requester, reason and CanDecide; creates nothing; 404 when no request is pending.
UPGetPlans	Validates the body (validator.validate("UPGetPlans"): product, variant, premium mode, term, age from DateOfBirth). Reads the variant (UPProductData.getVariant) and rule (getRule) and builds the PlanList directly from the tables: getPlanRows (UP_ProductPlanRelation: plan id, name, premium, commission) and getBenefitRows (UP_PlanBenefitRelation: code, description, sum insured) — no platform call. The premium shown here is therefore the table premium; the proposal premium is calculated by the platform rating <CODE>_PREM_CALC from the same tables, so the two stay equal as long as the data is consistent. Defaults of the policy skeleton (status, zero premium fields, flags, customer flags) come from UP_ApiConfig (PolicyStatusEffective, ZeroPremiumFields, CustomerFlagNo, ...). It does not call the platform and does not use shapeResponse, so CarrierCode / CarrierName are added with products.addCarrier before the return.
UPProposal	Rejects ProposalNo (409-style 400), validates, then UPProductData.prepareProposal: drops client PlanList and DuePremium (the platform rating builds plan, covers, premium and commission), removes server-owned fields and defaults OrgCode (UPSecurity), flattens PolicyPaymentInfoList into Payment* policy fields (the platform instalment rating would otherwise re-round the premium) and returns the payload. The API then calls proposalApi().newApplicationRequestBuilder() and returns shaper.shapeResponse, which rebuilds the payment structure, applies the commission switch and adds the insurer.
UPUpdateProposal	Same preparation as create. Loads the stored proposal by ProposalNo (404), checks channel, product match and that it is open (409), copies PolicyId, PolicyElementId and VersionSeq of the stored record into the payload, calls newSaveRequestBuilder() and then newPersistCalculateRequestBuilder() because the save drops the calculated plan row.
UPIssue	Loads the proposal (404), checks channel, product match (MISMATCH) and open state (409). When the optional request field Recalculate = Y it first runs persistCalculate on the stored proposal (not part of the public specification). Then newIssuePolicyRequestBuilder issues it and the shaped policy with the new PolicyNo is returned.
UPLoadPolicy	GET. Exactly one of policyNo / proposalNo (400); loads with code descriptions (withCodeDesc), 404 when missing, requireChannel, shapeResponse.
UPProposalReject	Loads the proposal (404), checks channel and open state (409), calls newRejectRequestBuilder with a RejectVo (proposal number and reason) and returns {status: SUCCESS (SuccessStatusText), message: <the reason>}.

API	Flow in code
UPDocGeneration / UPApplicationDocGeneration	Thin wrappers over <code>UPDocService.generate</code> : resolve the template name from <code>UP_DocTemplate</code> (product, sub-product, DocType), build the print data with <code>UPDocBuilder</code> (wording from <code>UP_DocWording</code> , benefit names from <code>UP_BenefitText</code> , formats from <code>UP_ApiConfig</code>), call the print service at <code>PrintUrl</code> (host checked against <code>AllowedHosts</code>) with the caller's token and return the PDF (or the print data when <code>PreviewData = Y</code>).
UPCancelCheck / UPCancelApprove	See section 8 for the complete sequence: <code>UPCancelService.check</code> and <code>decide</code> .
UPListMasterTable	Allows only tables listed in <code>UP_ApiConfig.MasterTables</code> ; returns all records or the one with <code>Id</code> (commission column removed).
UPCommissionQuery	Three modes: by <code>PolicyNo</code> (<code>UPDocBuilder.commissionOfPolicy</code>), by product configuration (rows of <code>UP_ProductPlanRelation</code> with premium, commission and rate), or for a period (search on the platform, load each policy, keep only the caller's channel, page totals). Page size is capped by <code>CommissionMaxPageSize</code> .
UPFeedFile	Searches policies with the platform query API (period, product, paging), loads each hit, keeps the caller's channel, shapes it and writes the columns defined in <code>UP_FeedColumn</code> (heading + source path, alternatives separated by <code> </code>) to an Excel sheet or to JSON. Capped by <code>FeedMaxRecords</code> / <code>FeedMaxPages</code> (400 <code>RANGE_T00_LARGE</code>).

6. Data and Configuration Reference

6.1 Tables

Table	Table id (MC)	Rows	Columns	Purpose	Read by
UP_ApiConfig	1188575596	113	Api, Scope, Key, Value	Key / value configuration of the code (Api, Scope, Key, Value). configMap(scope) merges * rows with product-specific rows.	UPDataUtil.cfg / cfgValue, every function
UP_FieldRule	1188575333	167	Api, Path, Label, Required, DataType, MinLength, MaxLength, Pattern, MinValue, MaxValue, CodeTable, CodeColumn, EnumValues, AppliesTo, WhenPath, WhenValue	Validation rules per API and field path: Required, DataType, MinLength, MaxLength, Pattern, MinValue, MaxValue, EnumValues, CodeTable + CodeColumn, WhenPath + WhenValue (conditional), AppliesTo.	UPValidator.validate
UP_ProductMaster	1188475672	11	Description, LOB, IMOProductCode, IsRenewable, IsActive, IsFlatRate, Premium, SumInsured, HasPlans, CarrierCode, CarrierProductCode, CarrierProductName, IsResident, BenefitRelationTableName, LobId, CarrierName	Product variants (IMOProductCode, CarrierProductCode = sub-product, name, LOB, CarrierCode, IsActive).	UPProductData.getVariant, carrierOf
UP_ProductPlanRelation	1188475676	57	IMOProductCode, IMOPlanCode, IMOPlanName, CarrierCode, CarrierProductCode, CarrierProductName, CarrierPlanCode, CarrierPlanName, IsActive, Premium, SumInsured, Commission, PremiumModeCode	Plans per variant with premium, commission, premium mode.	UPProductData, UPGetPlans, commission configuration view
UP_PlanBenefitRelation	1188474890	148	IMOProductCode, IMOPlanCode, CarrierCode, CarrierProductCode, CarrierPlanCode, BenefitCode, BenefitDesc, BenefitLevel, ParentBenefitCode, BenefitLimit, BenefitLimitDesc, IsMainBenefit, IsAdditionalBenefit, Notes, IsApplicable, LimitBasis, MaxDays	Covers and sums insured per plan.	UPProductData, UPGetPlans, documents

Table	Table id (MC)	Rows	Columns	Purpose	Read by
UP_Product Rule	1188475684	11	ProductCode, ProductSubcode, PremiumModes, MinTerm, MaxTerm, MinAge, MaxAge, MaxEntryAge, MaxExitAge, MandatoryDocuments, SinglePremiumRule, FreeLookDays, ProRataRefundPercent	Per variant: MinAge, MaxAge, MaxEntryAge, MaxExitAge, MinTerm, MaxTerm, PremiumModes, FreeLookDays, ProRataRefundPercent, mandatory documents.	UPValidator.entryErrors, UPCancelService.refundCalc
UP_DocTemplate	1188475689	22	DocumentCode, TemplateName, FileName, ProductCode, ProductSubcode, DocType	Output template name per product, sub-product and document type.	UPDocService
UP_DocWording	1188556760	27	ModeCode, Key, Value	Wording for premium modes and benefit limit bases (BASIS: *), English / Arabic.	UPDocBuilder
UP_BenefitText	1188575620	21	ProductCode, BenefitCode, NameEn, NameAr	Official English / Arabic benefit names per product.	UPDocBuilder
UP_FeedColumn	1188591810	29	Sequence, Heading, Source	Columns of the policy feed file (Sequence, Heading, Source).	UPFeedFile
UP_Channel	1188591925	1	ChannelCode, Description, IsActive	Registered channels (ChannelCode, Description, IsActive).	UPValidator (ChannelCode rule)
UP_Carrier	1188663015	1	CarrierCode, CarrierName, IsActive	Insurers (CarrierCode, CarrierName, IsActive); linked from UP_ProductMaster.CarrierCode.	UPProductData.carrierOf
UP_UserRole	1188710773	3	UserRoleKey, UserName, UserId, Role, DisplayName, Branch, Email, Mobile, IsActive	Staff roles: UserRoleKey (user ROLE), UserName, UserId (platform id), Role (MAKER / PROPOSAL_CHECKER / CANCELLATION_CHECKER), DisplayName, Branch, Email, Mobile, IsActive. Never exposed through master data.	UPAccess
UP_NotifyTemplate	1188710781	22	TemplateKey, Event, Recipient, Channel, Subject, Body, SmsTemplateCode, IsActive	Email / SMS texts per event, recipient and channel (TemplateKey, Event, Recipient, Channel, Subject, Body, SmsTemplateCode, IsActive).	UPNotify

The code maps a few logical names to the UP_ tables (UPDataUtil.readRecords): ProductMaster, ProductPlanRelation, UP_PlanBenefitRelation, ProductRule and DocTemplate. Tables are read once per request and cached in the request attributes, so a changed row is visible on the next request without any deployment.

6.2 Configuration Keys (UP_ApiConfig)

Every key with its current MC value and the functions that read it. Environment specific keys are marked in section 13.

Key	Value (MC)	Used in
AllowSelfApproval	Y	UPAccess, UPCancelService
AllowedHosts	portal-gw.insuremo.com	UPDataUtil
BeneficiaryShareTotal	100	UPValidator
BusinessCateCode	1	UPGetPlans
CancelFreeLookLabel	Free look	(read through a pattern or documented only)

Key	Value (MC)	Used in
CancelInstallmentFlag	Y	UPCancelService
CancelInstallmentType	1	UPCancelService
CancelPayMode	30	UPCancelService
CancelReasonCode	1	UPCancelService
CancelTypeFreeLook	31	UPCancelService
CancelTypeProRata	32	UPCancelService
CommissionDefaultPageNo	1	UPCommissionQuery
CommissionDefaultPageSize	50	UPCommissionQuery
CommissionMaxPageSize	50	(read through a pattern or documented only)
CustomerFlagNo	N	UPGetPlans
DashboardMaxDays	180	UPDashboard
DashboardMaxPolicies	60	UPWorklist
DateFormatDoc	dd/MM/yyyy	UPDataUtil
DayEndTime	T23:59:59	UPCommissionQuery, UPWorklist
DayStartTime	T00:00:00	UPCommissionQuery, UPWorklist
DecisionApprove	APPROVE	UPCancelService
DecisionReject	REJECT	(read through a pattern or documented only)
DocContentType	application/pdf	UPDocService
DocFileNamePattern	{docType}_{number}.pdf	UPDocService
DocTypeApplication	Application	UPApplicationDocGeneration, UPDocService
DocTypePolicy	Policy	UPDocService
EffectiveDateMinOffsetDays	0	UPValidator
EmailAccountName	NONE	UPNotify
EndoBasePath	/platform/endo/core/endo/v1	UPCancelService
EndoSearchModule	Endorsement	UPWorklist
EndoSearchPath	/platform/endo/v1/query	UPWorklist
EndoStatusIssued	300	UPWorklist
EndoStatusPending	120	UPWorklist
EndoStatusWithdrawn	600	(read through a pattern or documented only)
EndoTypeCancel	3	UPCancelService
EndoTypeCancelFilter	3	UPWorklist
FalseValue	N	UPDataUtil
FeedContentType	application/vnd.openxmlformats-officedocument.spreadsheetml.sheet	UPFeedFile
FeedFileMaxPageSize	50	(read through a pattern or documented only)
FeedFileName	Policy_Feed.xlsx	UPFeedFile
FeedMaxPages	40	UPFeedFile
FeedMaxRecords	2000	UPFeedFile
FeedSheetName	PolicyFeed	UPFeedFile
GatewayUrl	https://portal-gw.insuremo.com	UPPlatformClient
InstalmentAmount	0	UPShaper
InstalmentFeeSeq	1	UPShaper

Key	Value (MC)	Used in
InstalmentPayRate	1	UPShaper
InstalmentPeriodCount	1	UPShaper
InstalmentPeriodSeq	1	UPShaper
IsAutoAttachedForm	Y	UPGetPlans
IsPremiumCalcSuccess	Y	UPGetPlans
MasterTables	BranchCode, CustomerType, GenderCode, Id Type, InsMaritalStatus, Plan, Pre...	UPListMasterTable, UPValidator
MoneyFormat	#,##0.00	UPDataUtil
MoneyScale	2	UPDataUtil
MsgConfigError	The service is not configured correctly. Please contact support and...	UPErrorHandler
MsgForbidden	You do not have the role required for this action.	UPAccess
MsgInvalidNumber	A numeric value in the request is not valid.	UPErrorHandler
MsgNotOwner	Only the maker who created this proposal can change it.	UPAccess
MsgNotifyDryRun	Notification recorded (dry run, nothing was sent).	(read through a pattern or documented only)
MsgSelfApproval	The person who created an item cannot approve or reject it.	UPAccess
MsgServerError	The request could not be processed. Please contact support and quot...	UPErrorHandler
NonSingleTermYears	1	UPValidator
NotifyMode	DRYRUN	UPNotify
NumberFormat	#,##0.##	UPDataUtil
PatternMaxInput	500	UPValidator
PatternMaxLength	200	UPValidator
PercentBase	100	UPDataUtil
PlanLabelPrefix	PLAN-	UPDocBuilder
PlatformErrorMaxLength	300	UPCancelService
PolicyStatusEffective	2	UPGetPlans, UPShaper
PolicyStatusInforce	2	UPCancelService, UPWorklist
PortalUrl	https://portal.insuremo.com/uic-web/#!/p/sohar-insurance-portal	UPNotify
PrintExportType	pdf	UPDocService
PrintMode	sync	UPDocService
PrintStorage	not_store	UPDocService
PrintUrl	https://portal-gw.insuremo.com/mofo/1.0/print/v3/printtask/generate	UPDocService
ProposalLoadPath	/platform/proposal/v1/load	UPCancelService
ProposalStatusEntry	1	UPFeedFile
ProposalStatusIssued	3	UPCommissionQuery, UPFeedFile, UPValidator, UPWorklist
ProposalStatusIssued	3	UPCommissionQuery, UPFeedFile, UPValidator, UPWorklist
ProposalStatusPending	2	UPWorklist
ProposalStatusRejected	4	UPValidator, UPWorklist

Key	Value (MC)	Used in
ProposalStatusRejected	4	UPValidator, UPWorklist
RefundTypeFreeLook	FREE_LOOK	UPCancelService
RefundTypeProRata	PRO_RATA	UPCancelService
RequestStatusApproved	APPROVED	UPCancelService
RequestStatusPending	PENDING	UPCancelService
RequestStatusRejected	REJECTED	UPCancelService
ReturnCommission	Y	UPShaper
RiskElementCode	R00001	UPGetPlans, UPProductData
RoleCancellationChecker	CANCELLATION_CHECKER	UPAccess, UPCancelApprove, UPCancelDetail, UPWorklistApi
RoleEnforcement	Y	UPAccess, UPCancelService
RoleMaker	MAKER	UPAccess, UPCancelCheck, UPCancelDetail, UPDashboard, UPProposal, UPUpdateProposal, UPWorklistApi
RoleProposalChecker	PROPOSAL_CHECKER	UPAccess, UPIssue, UPProposalReject, UPWorklistApi
RoundingMode	HALF_UP	UPDataUtil
SearchModule	Policy	UPCommissionQuery, UPFeedFile, UPWorklist
SearchPath	/platform/proposal/v1/queryPolicy	(read through a pattern or documented only)
SearchSortField	index_time	UPCommissionQuery, UPFeedFile, UPWorklist
SearchSortType	desc	UPCommissionQuery, UPFeedFile, UPWorklist
SequenceNumberStart	1	UPGetPlans, UPShaper
ShareDocPolicy	SHARE_POLICY	UPShare
ShareDocProposal	SHARE_PROPOSAL	UPShare
ShareDocQuotation	SHARE_QUOTATION	UPShare
ShareMaxDetailLength	80	(read through a pattern or documented only)
SinglePremiumModeCode	1	UPValidator
SmsAccountName	NONE	UPNotify
SuccessStatusText	SUCCESS	UPProposalReject
TemplateVersion	1	UPDocService
TrueValue	Y	UPDataUtil
WorklistMaxRows	1000	UPWorklist
WorklistPageSize	20	UPWorklist
ZeroPremiumFields	AdjustedPremium,AnnualPremium,BeforeVa tPremium,GrossPremium,Standar...	UPGetPlans, UPShaper
ZeroPremiumValue	0	UPGetPlans, UPShaper

6.3 Field Rules (UP_FieldRule)

API	Rules
UPApplicationDocGeneration	3
UPCancelApprove	5
UPCancelCheck	5
UPCancelDetail	3
UPCommissionQuery	9
UPDashboard	6

API	Rules
UPDocGeneration	6
UPFeedFile	8
UPGetPlans	8
UPIssue	3
UPListMasterTable	3
UPLoadPolicy	3
UPProposal	40
UPProposalReject	4
UPShare	12
UPUpdateProposal	41
UPWorklist	8

A rule row is evaluated by `UPValidator.validate`: Path uses indexes (`PolicyLobList[0].PolicyRiskList[0].InsuredName`); Required Y/N; `DataType` string / integer / decimal / date / enum; `Pattern` is a regular expression (length and input size are capped by `PatternMaxLength` / `PatternMaxInput`); `CodeTable` + `CodeColumn` check the value against a table (`UP_Channel`, `UP_Carrier`, `BranchCode`, ...); `WhenPath` + `WhenValue` make the rule conditional (for example `PolicyNo` is required when `DocType = Policy`); `AppliesTo` limits a rule to a product. Changing a rule is a row change and needs no deployment.

6.4 Security Baselines (UPSecurity)

Control	Baseline in code	Optional override key in UP_ApiConfig
Blocked characters in any text value	control characters and <code>< > \$ { } \ ^ ~ ; `</code>	BlockedCharacters
Free-text fields	InsuredName, Address, BeneficiaryName, CancelReason, OtherCancelReason, ProposalRejectDesc	FreeTextFields
Free-text allow-list	letters of any script, digits, space and <code>. , ' - / @ + ()</code>	FreeTextPattern
Server-owned fields removed from a proposal	PolicyNo, PolicyId, PolicyElementId, ProposalStatus, PolicyStatus, AgentCode (v1.2: the maker stamp)	ServerOwnedFields
Default OrgCode	1	OrgCode

These are the only values that have a built-in default; a missing override key never fails a request. Moving them fully into tables (and treating a missing key as UP-CONFIG) is on the backlog.

7. Platform Configuration (Products)

7.1 What Exists on the Platform

For each of the seven product codes the platform holds a tech product `<CODE>_TECH` and a market product `<CODE>` (ids in the deployment plan), the policy data dictionary tree (Policy, PolicyLob, PolicyRisk R00001, Plan, PolicyCoverage, Beneficiary), a premium rating definition `<CODE>_PREM_CALC` bound to the market product, the plan and cover data in the `UP_` tables, and the output templates named in `UP_DocTemplate` (22 rows).

7.2 Data Dictionary Bindings

The new products bind only the fields the flows need (24 policy-level, 17 tech-level, 8 risk-level, 4 plan, 6 coverage, 4 beneficiary fields) at product level (RecordUsage 5). PA001 binds far more (408 policy-level fields). The cancellation endorsement rating additionally reads delta fields; `fix_endo_fields.py` bound the ones the platform reported as missing (`StandardGrossPremiumDelta`, `AnnualPremiumDelta`, `ShortRate` on the risk and coverage objects) on the new tech products only. The script never creates `VAT` (there is no VAT in this solution) and refuses to touch any context that is not one of the seven tech products.

7.3 How the Products Were Built

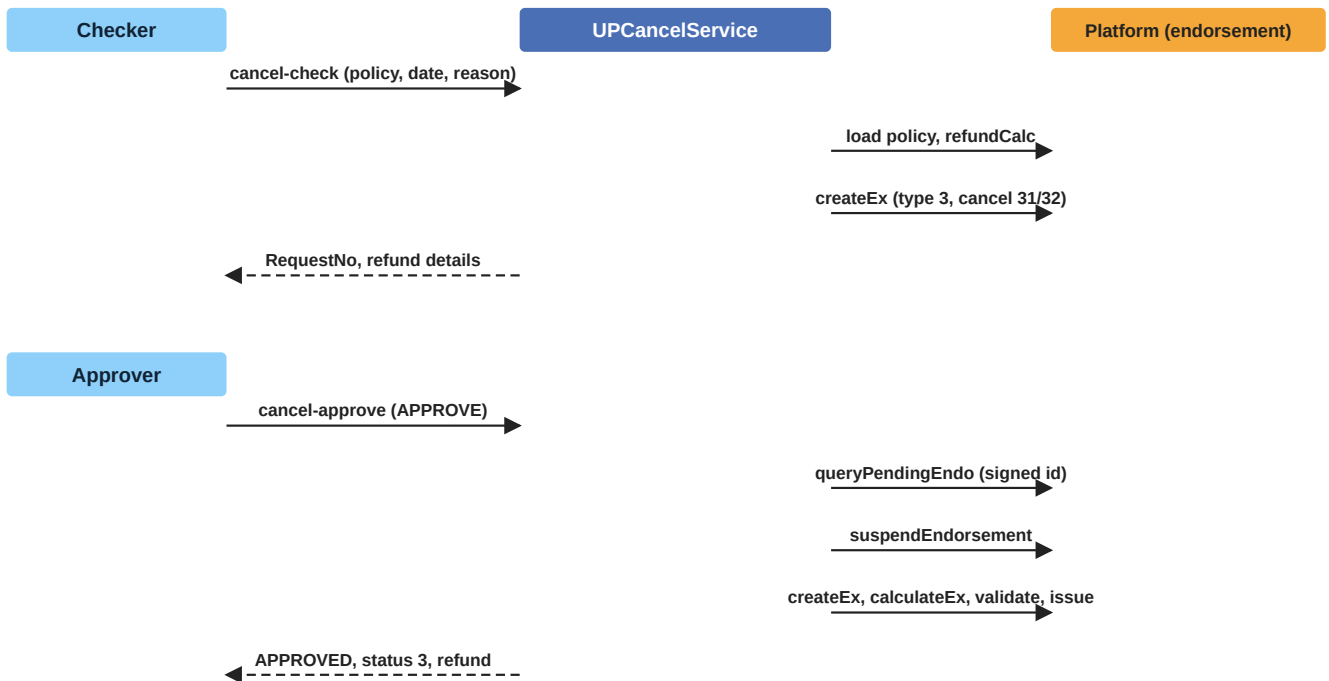
Step	Tool
Tech and market products, tree, DD fields, code tables	<code>build_product.py <spec.json> <step></code> with <code>spec_<CODE>.json</code>
Rating definition per product	<code>rating_build.py <CODE> --confirm</code> (copy of <code>PA001_PREM_CALC</code> bound to the market product), patched by <code>patch_rating.py</code> / <code>patch_rating2.py</code>
Tables and rows	<code>make_tables.py</code> , <code>data_rows.py</code> , <code>uptab.py</code> (create table, save rows), <code>make_rules.py</code> (field rules), <code>stage_*_rows.py</code> (config keys)
Output templates	<code>make_templates2.py</code> builds the bilingual HTML templates from the form designs; <code>gen_all_tpl.py</code> / <code>os_gen.py</code> upload and test them
Cancellation fields	<code>fix_endo_fields.py</code> , <code>bind_cancel_field.py</code> (optional <code>IsAdminChange</code> , not used)
PA001 guard	<code>snapshot.py before after</code>

8. Business Algorithms

8.1 Premium Mode, Term and Age

UPValidator.entryErrors reads the variant's UP_ProductRule row: the premium mode must be in PremiumModes; single premium (SinglePremiumModeCode) needs a term between MinTerm and MaxTerm, other modes need NonSingleTermYears (1); the age at the effective date must be within MinAge / MaxAge and MaxEntryAge, and age + term within MaxExitAge. PremiumModeCode is mandatory where the plans of a variant differ by mode (UPPA001, TL001, CI001, LP001). Premium and commission are never taken from the request: the platform rating calculates them from the plan tables.

8.2 Cancellation (Checker / Approver)



When the platform refuses a step the fresh endorsement stays pending: the answer is HTTP 409 and the request can be approved again or rejected.

Figure 2: Cancellation — Checker and Approver sequence in UPCancelService

The request is the platform's own pending cancellation endorsement; its number is the RequestNo. From v1.2 the Checker step is the **Maker** role and the Approver step the **Cancellation Checker** role (the code and routes are unchanged); the requester cannot decide their own request; cancel-detail shows a pending request without creating one; and the steps send alerts (section 16).

Checker — UPCancelService.check

1. resolvePolicy: by PolicyNo or ProposalNo (a proposal must be issued: 409), requireChannel, the product must be enabled (UP_ProductMaster).
2. Policy must be in force (status PolicyStatusInforce); a cancelled policy is 409.
3. pendingCancellation: queries the pending endorsement with the **signed policy id** (id,signature) returned by the platform's own load (ProposalLoadPath), sent once-encoded. Another kind of pending endorsement blocks the request (409). An existing cancellation request is shown again with the date it was created with.
4. refundCalc(policy, date): free look when elapsed days <= FreeLookDays (refund 100%), otherwise premium x unexpired days / policy days x ProRataRefundPercent (rounded MoneyScale, RoundingMode).
5. createRequest: POST EndoBasePath/createEx with EndoType 3, CancelType 31 (free look) or 32 (pro-rata), reason, payment info; for free look the effective date is the policy start.
6. Response: RequestNo, RequestStatus PENDING, PolicyInfo (with insurer), PolicyStatus, PremiumDetails, RefundDetails.

Approver — `UPCancelService.decide`

1. Same lookup; `RequestNo` must be the pending endorsement (404 otherwise, also on a second approval).
2. REJECT: `suspendEndorsement`, answer REJECTED.
3. APPROVE: suspend the saved (thin) request, create a **fresh** endorsement with the same terms (`createEx`), then `calculateEx`, `validate`, `issueEndorsement`. The saved request is replaced because `calculateEx` needs the full `NewPolicy`, which only `createEx` returns.
4. If the platform refuses a step the exception text is capped (`PlatformErrorMaxLength`) and returned as 409
The platform refused the cancellation of `<policy>`; the request is still pending as `<EndoNo>`.
5. Success: status 3, `RefundPremium` (same calculation as the Checker), `EndorsementNo`, `RequestStatus` APPROVED.

8.3 Platform Quirks That Shaped the Code

Quirk	Effect / handling
Endorsement service needs the signed policy id (<code>id,signature</code>) and rejects a double-encoded id	Signed id taken from the platform load and sent once-encoded in <code>pendingCancellation</code> .
<code>calculateEx</code> needs the full <code>NewPolicy</code> ; a saved pending endorsement is thin	On APPROVE the saved request is suspended and a fresh <code>createEx</code> is used.
The platform REST client throws its own business exception	<code>UPPlatformClient</code> returns <code>{status, body}</code> ; the approver answers 409 instead of 500.
A policy created without <code>OrgCode</code> gets -1; <code>validate</code> of the endorsement checks <code>OrgCode</code> against code table <code>PubBranch</code> (only value 1)	<code>UPSecurity.defaultOrgCode</code> sets 1 when it is missing; callers send <code>OrgCode</code> 1.
A policy status sent by the caller made the platform fail (500 / 502); a caller policy number was accepted	<code>UPSecurity.stripServerOwned</code> removes <code>PolicyNo</code> , <code>PolicyId</code> , <code>PolicyElementId</code> , <code>ProposalStatus</code> , <code>PolicyStatus</code> .
The cancellation rating reads delta fields that were not bound on the new products	<code>fix_endo_fields.py</code> bound them on the seven tech products (section 7.2).
The platform save drops the calculated plan row	<code>UPUpdateProposal</code> calls <code>persistCalculate</code> after save.
Platform instalment rating re-rounds the premium	Payment info is flattened to <code>Payment*</code> fields and rebuilt in <code>UPShaper.restorePayment</code> .
Responses contain a <code>TempData</code> block with masked / encrypted copies of ID and mobile	Not part of the contract; flagged for review (section 14).
Wrong HTTP method returns 422 from the gateway, not from the code	Documented in the specification.

9. Errors, Logging and Tracing

Situation	Exception thrown in code	HTTP / code
Validation problems	<code>IllegalArgumentException("VALIDATION:" + json)</code> via <code>util.throwValidation</code>	400 UP-VALIDATION with <code>errors[]</code>
Simple bad request	<code>util.fail(msg) -> IllegalArgumentException</code>	400 UP-400
State conflict	<code>IllegalArgumentException("CONFLICT:...")</code>	409 UP-409
Bad number	<code>NumberFormatException</code>	400 UP-400, text <code>MsgInvalidNumber</code>
Missing configuration / table row	<code>IllegalStateException</code>	500 UP-CONFIG, text <code>MsgConfigError</code> ; detail in the log
Platform business error	<code>ApiException</code> (message sanitised by <code>isInternalText</code>)	status of the platform, its code; internal text replaced by <code>MsgServerError</code>
Missing role / maker-checker breach	<code>IllegalArgumentException("FORBIDDEN:...")</code> from <code>UPAccess</code>	403 UP-403, texts <code>MsgForbidden</code> , <code>MsgSelfApproval</code> , <code>MsgNotOwner</code>
Not found	<code>errorHandler.notFound(api, msg)</code>	404 UP-404
Anything else	<code>Exception</code>	500 UP-500, generic text; full exception logged with the trace id

Tracing: every error body carries `trace_id`. Find the call in the platform log with `imo log` query using that trace id (application logs of the iComposer runtime); UP-CONFIG messages name the missing key only in the log. Do not add `println` or secrets to the code; use the slf4j logger.

10. How-To Recipes

Add or change a validation rule

1. Read the rows of `UP_FieldRule` (`imo config datatable record-list --id 1188575333`).
2. Add a row (`record-batch-save`, a new unique Id) or change one (`record-save` with its `RecordId`): `Api`, `Path`, `Label`, `Required`, `DataType`, `limits`, `Pattern`, `CodeTable`.
3. Test the API with a valid and an invalid value; no deployment is needed.
4. Update the specification (`api_spec_data.py` reads the rules) and the policy model.

Rule that needs code (cross-field, product rule)

1. Edit `UPValidator.entryErrors / validate` (keep one `util.fieldError(path, code, message)` per problem).
2. Push `UPValidator`; push nothing else unless a signature changed.
3. Add a test case (`run_tests.py`, business cases) and a line in the specification.

Change free look days or refund percent

1. Edit `FreeLookDays / ProRataRefundPercent` of the variant row in `UP_ProductRule` (`record-save`).
2. Check with `cancel-check` on a test policy; update the specification section 6.2 if the value differs from the documented 15 / 80.

Add a plan or change a premium

1. Add or change rows in `UP_ProductPlanRelation` and `UP_PlanBenefitRelation`; add the plan id to the `Plan` code table if it is new.
2. Run `Get Plans` and `Create Proposal` for the plan and compare the premium with the table (the rating reads the tables).

Add a product variant (sub-product)

1. Add `UP_ProductMaster`, `UP_ProductRule`, plan and cover rows, `UP_DocTemplate` rows and the output templates (`make_templates2.py`).
2. Add test samples (`samples/req/<TAG>_proposal.json`) so `ref_flow.py` can run the variant.

Add a new product code

1. Create the tech and market product, the DD tree and bindings and the rating definition with `build_product.py` and `rating_build.py` (section 7.3).
2. Bind the cancellation delta fields with `fix_endo_fields.py <TAG>`.
3. Add the table rows as for a variant; add the code to the product lists of the tooling (`products.json`, test scripts).

Add a channel

1. Add a row to `UP_Channel` (`ChannelCode`, `Description`, `IsActive Y`). The `ChannelCode` rule checks the value against this table.

Add or change the insurer

1. Add a row to `UP_Carrier`; set `UP_ProductMaster.CarrierCode` of the variants to it. Responses show the name from the table; a product whose carrier row is missing or inactive returns `UP-CONFIG`.

Add a master (code) table to Master Data

1. Append the table name to the value of `UP_ApiConfig.MasterTables` (`record-save` of the row with its `RecordId`).

Add a field to a response

1. Policy-level: set it in `UPShaper.shapeResponse` (or the API for `Get Plans`, which does not use the shaper).
Cancellation: `UPCancelService.policyInfo` and the `out` maps. Commission: `UPDocBuilder.commissionOfPolicy`. Feed: add a `UP_FeedColumn` row.
2. Add the field to the specification (`api_spec_data.py` reads recorded responses: re-run `ref_flow.py` for one plan per product).

Add a new API

1. `imo icomposer create api ...` (path `/up/<name>`), copy the pattern of a small API (readInput, validate, call, shape, errorHandler).
2. Add `UP_FieldRule` rows for the new API name, push, add cases to `run_tests.py` and `security_probe.py`, document in the specification.

Change a message text or format

1. Edit the `UP_ApiConfig` value (`MsgServerError`, `MsgConfigError`, `MsgInvalidNumber`, `DateFormatDoc`, `MoneyFormat`, ...).

Move to another environment

1. Change `AllowedHosts`, `GatewayUrl`, `PrintUrl` (section 13) and re-check the ids listed in the deployment plan.

Add a user or change a role

1. Section 19.2: rows in `UP_UserRole` (`record-batch-save` / `record-save`); no deployment.

Change a message text or add an event

1. Edit the `Subject` / `Body` of the row in `UP_NotifyTemplate` (`record-save`). For a new event add rows (`TemplateKey` = `Event|Recipient|Channel`) and call `notify.fire("EVENT", notify.paramsOf(policy, extra), customer, makerUserId, "")` from the API; add the placeholders to `paramsOf` if needed.

Send real email / SMS

1. Section 16.5.

Add a menu item or screen to the portal

1. Add a `<section class="view" id="v_x">` and an entry in `NAV` (roles list or null for all); load its data in `go()`; call the API through `api()`; add a browser check to `flow2.js`; push with `imo uic page push`.

Add a column to a queue

1. Add the field to `UPWorklist.policyRow`, push `UPWorklist`, then add a column object to `colsProposal` / `colsCancel` in the portal.

Turn role enforcement on or off, or allow self approval

1. `python3 setcfg.py RoleEnforcement Y|N` and `AllowSelfApproval Y|N` (section 19.1).

Apply the Sohar brand

1. Edit the variables in the first `:root` block of `index.html` (colours, font) and the text badge; push the page.

11. Test Tooling

All scripts live in `uponly-build` and read the token and gateway from `env.sh` (`source env.sh` sets `B`, `GW`, `TOK`; the token is fetched with `imo auth prepare --profile portal:uponly`). Never commit tokens.

Script	What it does
<code>carrier_check.py</code>	One flow per product (first plan): <code>CarrierCode</code> / <code>CarrierName</code> in every API, wrong <code>CarrierCode</code> , master data and feed file; output <code>carrier/results.json</code> .
<code>ref_flow.py</code>	Runs the 13-step flow (get plans ... cancel approve, load cancelled policy) for every plan of a variant; <code>ONLY_PLAN=1</code> limits it to one plan; output in <code>samples/ref/<TAG>_P<plan>/</code> .
<code>refresh_devdata.py</code>	Rebuilds <code>devdata/assets.json</code> and <code>devdata/tables.json</code> offline for this guide.
<code>role_check.py</code>	v1.2: 35 live checks of roles, maker-checker rule, queues, cancellation detail, notifications (dry run), share, dashboard; switches <code>UP_UserRole</code> rows and <code>AllowSelfApproval</code> during the run and restores them.
<code>run_tests.py</code>	Generates and runs the API test suite (1,085 cases: field validation, business rules, state transitions, cancellation incl. refund checks per product, master data, feed file, robustness). Results in <code>tests/results/</code> and <code>tests/results.json</code> .
<code>security_probe.py</code>	Live security probe (token tampering, channel isolation, injection, tampering, resource abuse, headers, errors, request hygiene); output <code>security/results.json</code> .
<code>setcfg.py</code>	Changes <code>UP_ApiConfig</code> values with <code>record-save</code> and prints the old value.
<code>snapshot.py</code>	Read-only snapshot of PA001 related objects; compare <code>before</code> and <code>after</code> to prove PA001 is unchanged.
<code>workflow_setup.py</code>	v1.2 set-up: creates <code>UP_UserRole</code> , <code>UP_NotifyTemplate</code> and the first configuration keys / field rules (<code>workflow_setup2..5.py</code> add the later keys and rules).

Collections: `make_sohar_collection.py` (single Sohar Bank collection, one flow per product), `make_plan_collections.py` (one collection per plan with recorded failure examples), `make_cases_by_product.py`. Reports: `gen_report.py`, `make_excel.py`. Documents: `build_policy_model.py` (workbook), `spec_content.py` + `render_spec.py` / `render_docx.py` (specification), `dev_content12.py` + `dev12_sections.py` (this guide; the v1.1 generator is `dev_content.py`). Browser tests of the portal (`flow2.js`, `login.js`, `run.js`, Playwright) are described in section 17.6.

All scripts in the build folder

Script	Purpose (first docstring line)
<code>api_spec_data.py</code>	One structured specification per API (inputs with M / O / C, outputs with A / C), shared by the workbook and the specification document. Inputs come f
<code>bind_cancel_field.py</code>	Adds ONE field binding, <code>IsAdminChange</code> (string), to the cancellation endorsement object (id 300224988) for the Uponly TECH products only. It never touc
<code>build_api_docs.py</code>	External API documentation (Markdown) + OpenAPI 3.0 spec for the Uponly insurance APIs. Internal details (tenant tables, configuration keys, code name
<code>build_policy_model.py</code>	Builds <code>Afillar_PolicyModel_v0.4.xlsx</code> in the layout of the PA policy model workbook (v0.1), extended to all Uponly products and APIs.
<code>build_product.py</code>	Component-path fields are created as temporary (<code>IsTemp=Y</code> , not stored). Make them persisted like Uponly's.
<code>carrier_check.py</code>	One flow per product (first variant, first plan): <code>CarrierCode</code> / <code>CarrierName</code> must be returned by every API. Writes <code>carrier/results.json</code> .
<code>carrier_setup.py</code>	Creates the insurer table <code>UP_Carrier</code> and the rows that expose <code>CarrierCode</code> / <code>CarrierName</code> (field rules, feed columns, master table list). Idempotent: id
<code>code_facts.py</code>	Facts extracted from the real source files, tables and scripts, so that the developer guide cannot drift from the code.
<code>dev12_sections.py</code>	Developer guide v1.2: the content added after v1.1 (handover risks, workflow / roles / dashboard, notifications, portal, PA001 and iHub, runbook). Eac
<code>dev_content.py</code>	Developer guide content (block list). Reference sections are generated from the real source, tables and scripts.

Script	Purpose (first docstring line)
dev_content12.py	Developer guide v1.2 content (block list). Reference sections are generated from the real source, tables and scripts; sections 1.4, 1.5 and 15 to 19 a
fix_endo_fields.py	Admin script: makes the platform's cancellation calculation pass for an Uponly product by binding the fields it reports as 'not bound'. usage: fix_end
make_plan_collections.py	One Postman collection per plan; one folder per API; real flow request/response + recorded failure examples.
make_ref_postman.py	Builds the TrialX collection in exactly the structure of the UPONLY_UAT reference collection.
make_rules.py	Creates UP_FieldRule, UP_ApiConfig, UP_DocWording, UP_BenefitText in Uponly and loads their rows.
make_sohar_collection.py	Single 'Sohar Bank' collection: one example per product, token mapped, values parameterized.
make_tables.py	Create the UP_data tables in Uponly (definitions and rows copied from the tested TrialX tables).
make_templates2.py	Bilingual (english, arabic) paragraphs from the long text rows of the application docx.
md_to_docx.py	Minimal Markdown -> docx (headings, paragraphs, bullets, tables, bold, code spans). usage: md_to_docx.py in.md out.docx
model_data.py	Collects the facts for the policy model workbook and the external API documentation from the build folder (product tables, field rules, code tables, s
os_gen.py	usage: os_gen.py TEMPLATE_NAME html_file data.json out.pdf (uploads and generates)
rating_build.py	Create <CODE>_PREM_CALC (copy of Uponly PA001_PREM_CALC) bound to a market product in the current tenant. usage: rating_build.py <PRODUCTCODE> [--conf
ref_flow.py	Runs the PA001-style 11 step flow for every plan of every variant, using the request bodies of the UPONLY_UAT reference collection as the template, an
refresh_devdata.py	Rebuilds devdata/assets.json (server metadata saved by the last push of every asset) and devdata/tables.json (v1.1 data plus the rows written by workf
render_docx.py	DOCX version of the specification (same content blocks): real heading styles, TOC field, repeating table header rows, page numbers.
render_spec.py	Renders the specification content to PDF (HTML + Chromium, TOC page numbers from a first pass) and DOCX (python-docx), in the layout of the InsureMO r
role_check.py	Role / maker-checker / notification checks on the live MC tenant. One account holds all three roles in UP_UserRole, so roles are switched by (de)activ
run_tests.py	Stored copy: large responses are cut to keep the result files small (the live copy keeps the full body for the follow-up steps).
security_probe.py	Live security (VAPT style) probe of the deployed Uponly APIs in the test tenant. Writes security/results.json. Each check: id, category, title, what w
setcfg.py	usage: setcfg.py Key Value [Key Value ...] - changes UP_ApiConfig values (record-save) and prints the old values.
snapshot.py	Read-only snapshot of PA001 related Uponly objects. usage: snapshot.py before after
spec_content.py	Content model of the API specification (list of blocks) in the structure of the InsureMO reference specification.
split_util.py	Splits UPDataUtil into focused functions and rewrites the callers. Usage: split_util.py <src-root> <out-root>
stage_a_rows.py	Stage A: new configuration rows (no defaults in code) + UP_FeedColumn.
stage_cd_rows.py	Stages C + D: ChannelCode rules on every API, cancel-check / cancel-approve rules and config keys, ChannelCode feed column.
status_cancel_spike.py	Spike: can an issued Uponly policy be set to Cancelled through the platform's own proposal update calls? Works on ONE throwaway Uponly policy only (th
uat_content.py	UAT deployment plan content (block list) + data for the checklist workbook.
uptab.py	Small helpers to create / extend Uponly data tables (same approach as make_rules.py).
workflow_setup.py	Maker / Checker workflow set-up: creates UP_UserRole and UP_NotifyTemplate and the config keys / field rules the new APIs need. Idempotent for tables;

12. Test Procedure for a Change

1. Before: `snapshot.py before` (PA001 guard) and copy `ic` to a backup folder.
2. Make the change; push with `imo icomposer push current` (the compile check must say `Compiled: yes`).
3. Run the affected cases (`run_tests.py all <filter>`) and, for response changes, `ref_flow.py` for one plan per product and `carrier_check.py`.
4. For anything touching input handling or errors run `security_probe.py`.
5. After: `snapshot.py after` and compare with `before` (PA001 and PA001_TECH hashes identical).
6. Update the documents (specification, policy model, this guide) and the collection (`make_sohar_collection.py`), then run the collection in Newman.
7. Rollback: push the saved files back; for tables restore the saved row.

13. Environments and Deployment

The complete inventory, ordered steps, verification and rollback for deploying to UAT are in the separate document *Afillar API UAT Deployment Plan*. In short: 13 functions and 18 APIs (v1.2), 14 `UP_` tables with their rows (v1.2 added `UP_UserRole` and `UP_NotifyTemplate`), the product configuration on the platform, the output templates and the environment settings. The keys that differ per environment are `AllowedHosts`, `GatewayUrl` and `PrintUrl`; the ids of products, data tables and templates are tenant specific and must be re-resolved, not copied. **The UAT plan was written for v1.1: for v1.2 also deploy the portal page, the two new tables, the new keys and rules, set up users and roles, and the SNS accounts (section 19).**

14. Known Limitations and Backlog

Item	Description	Suggested action
Role / branch access	Any authenticated user of a channel can call any API; the ChannelCode check isolates channels only.	Access table per API and role (deferred by decision).
Security baselines in code	UPSecurity constants with optional override keys.	Add the override keys / UP_FieldRule patterns and make a missing key UP-CONFIG.
Gateway findings	Wildcard CORS, no rate limiting, OPTIONS 500, attack-like query strings hold the connection.	Platform team.
ID number and TempData in responses	The ID number is returned in clear and a TempData block with encrypted copies is part of the platform body.	Decide on masking; strip TempData in UPShaper.
Idempotency and audit	No idempotency key; no audit record of who approved a cancellation beyond the platform endorsement.	Idempotency key on create / approve; audit table.
Unit tests	Only black-box API tests exist.	Data-driven tests for pure functions (refundCalc, entryErrors, scan).
Per-plan collections	52 per-plan collections were not regenerated after the insurer fields were added (still valid; examples lack the two fields).	Re-run make_plan_collections.py after ref_flow.py for all plans.
PDF templates	The insurer name is not printed on the documents.	Add CarrierName to the print data and the templates.
v1.2 items	See the consolidated list in section 19.6 (notifications live, sign-in, portal URL, role checks on feed and master data, maker stamp, dashboard accuracy, documents).	Section 19.6.

14.1 Troubleshooting

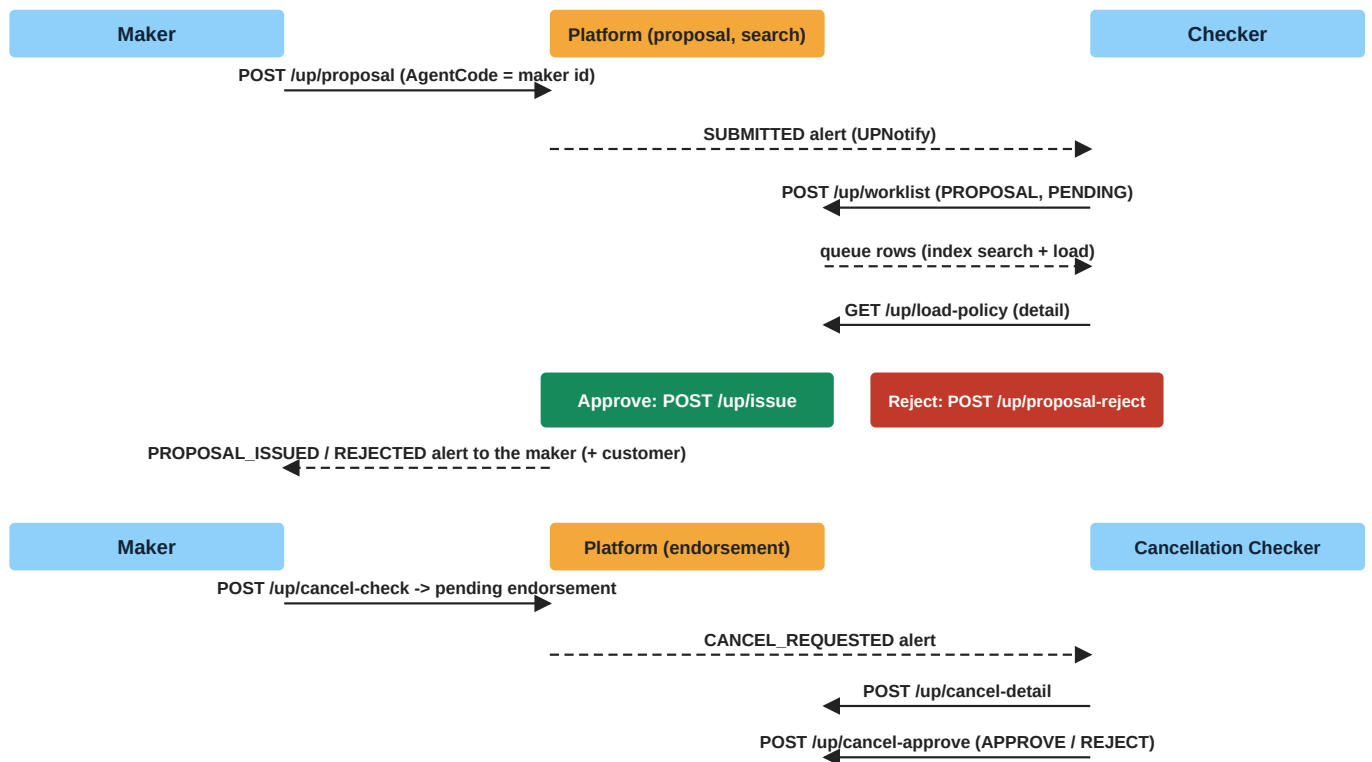
Symptom	Likely cause	Check
HTTP 500 UP-CONFIG	A configuration key, table row (carrier, channel, product) is missing	Log of the trace id names the key / table
Cancel approve returns 409 platform refused	Platform validation of the endorsement (for example OrgCode -1 on an old policy)	Text after the policy number; run probe_approve.py <PolicyNo> to see each step
Get Plans returns no plan	No active plan row for the variant and premium mode	UP_ProductPlanRelation
400 UNKNOWN_VALUE on a code	Value missing in the code table	imo config codetable data --name X
Push fails to compile	Callee not pushed yet or a Groovy static type error	Push order (section 3.4)
401 from every call	Token expired	Get a new token
Document not generated	Template missing or print host not in AllowedHosts	UP_DocTemplate, PrintUrl, AllowedHosts

14.2 First-Day Checklist for a New Developer

1. Install imo, log in to the MC tenant (imo auth login --profile portal:uponly).
2. Clone the source export, read sections 2 and 4 of this guide.
3. source env.sh, run carrier_check.py (one flow per product) to see the APIs work.
4. Open UPCancelService.groovy and UPValidator.groovy next to sections 8 and 6.3.
5. Make a harmless change in a copy tenant or a branch: add a UP_FieldRule row and see the 400.
6. Read the specification and the policy model workbook (API-Field sheets) for the contract.

15. Workflow, Roles and Dashboard

15.1 Overview



Rule: the person who created the proposal / request cannot approve or reject it (UPAccess.requireNotCreator, switch AllowSelfApproval).

Figure 3: Maker-checker flow for proposals and cancellations

Bank staff use the portal in three roles. The **Maker** creates quotations and proposals and raises cancellation requests. The **Proposal Checker** approves (issues) or rejects submitted proposals. The **Cancellation Checker** approves or rejects cancellation requests. No new workflow state is stored by our code: the platform already holds it. A proposal waiting for approval is a platform proposal with ProposalStatus 2. A cancellation request is the platform's pending cancellation endorsement (status 120). The queues are searches over that state, so there is nothing to keep in step and nothing to clean up.

15.2 Roles and Identity

The caller is taken from the platform token inside iComposer: `IComposerAppContext.getUserName()` and `IComposerAppContext.getIComposerCurrentUser()` (an `AppUser` with `userId`, `userName`, `realName`, `email`). The request body is never trusted for identity. The platform's own role list (`userRoleIds`) is empty for these users, so roles come from the table `UP_UserRole` (row per user and role; matched by user name, case-insensitive, or by platform user id). `UPAccess` offers:

Method	Behaviour
<code>me()</code>	UserName, UserId, DisplayName, Email, Branch, Roles[], RoleEnforcement, AllowSelfApproval. Roles is omitted from the JSON when empty.
<code>require(roles)</code>	Returns the caller; HTTP 403 UP-403 unless the caller holds one of the roles. Does nothing when RoleEnforcement is not Y.
<code>requireAny()</code>	Any of the three roles (read access to policy data).
<code>requireNotCreator(caller, creatorId)</code>	Maker-checker rule: 403 when the caller created the item, unless AllowSelfApproval = Y or enforcement is off.
<code>requireCreator(caller, creatorId)</code>	A maker may change only what they created (update proposal). An empty creator id (older proposals) is allowed.
<code>holdersOf(role) / contactOf(userId)</code>	Recipients for alerts: active rows of a role, or the row of one user id.

Role names are fixed strings used in four places and must stay in step: the config keys `RoleMaker`, `RoleProposalChecker`, `RoleCancellationChecker` (values `MAKER`, `PROPOSAL_CHECKER`, `CANCELLATION_CHECKER`), the `Role` column of `UP_UserRole`, the `Recipient` column of `UP_NotifyTemplate`, and the `ROLE` constants of the portal.

Which API needs which role

API	Roles	Extra rule
UPProposal (create)	Maker	Stamps the maker id on the proposal (15.3).
UPUpdateProposal	Maker	Only the maker who created the proposal (<code>requireCreator</code>).
UPIssue	Proposal Checker	Not the creator of the proposal.
UPProposalReject	Proposal Checker	Not the creator; a reason is required (field rule).
UPCancelCheck	Maker	Creates the pending request; alert to the Cancellation Checkers.
UPCancelDetail	Maker or Cancellation Checker	Read only; <code>CanDecide</code> tells whether the caller may decide.
UPCancelApprove	Cancellation Checker	Not the person who raised the request (checked inside <code>UPCancelService.decide</code> against the endorsement's <code>DataEntryUserId</code>).
UPWorklistApi	PROPOSAL: Proposal Checker. CANCELLATION: Cancellation Checker or Maker. MINE: Maker.	Type decides the role.
UPDashboard, UPSHare, UPGetPlans, UPLoadPolicy, UPDocGeneration, UPApplicationDocGeneration, UPCommissionQuery	any of the three roles	A caller who is only a Maker sees only their own figures in the dashboard.
UPMe	none	Always answers; used by the portal to build the menu.
UPListMasterTable, UPFeedFile	no role check	Left open as before. Add <code>requireAny()</code> (or a checker role) to <code>UPFeedFile</code> if the feed must be restricted: it contains policy data.

15.3 The Maker Stamp (AgentCode)

The platform search index cannot be filtered by the user who created a proposal. Of the indexed fields only `AgentCode` can carry the maker id. `UPProposal` therefore writes `AgentCode` = the caller's platform user id on every new proposal, and `UPUpdateProposal` keeps the stored value. `AgentCode` is in the server-owned field list of `UPSecurity` (`serverOwnedBaseline`), so a value sent by a caller is removed before the stamp is applied. The queue "My submissions" is a search with `AgentCode` = the caller's id; the maker-checker rule compares the same field with the checker's id. The id is mapped to a name through `UP_UserRole.UserId` (otherwise the platform's `DataEntryUserRealName` is shown).

Consequences: proposals created before v1.2 have no maker id (they appear in the checker queue but never in "My submissions"); if the bank later needs real sales agent codes, another field and a new search condition are needed; the platform does not validate `AgentCode` against a code table (verified).

15.4 Queues

Proposal queue (type PROPOSAL and MINE)

- `UPWorklist.proposalList` runs `ProposalSdkClient...newQueryPolicyRequestBuilder` once per enabled product (the index supports only one exact product code per query), with `ProposalStatus` and the date window on `ProposalDate`, then merges the hits by date, takes the requested page and loads each proposal of the page (channel check, maker name, plan, premium).

- Status codes from UP_ApiConfig: ProposalStatusPending 2, ProposalStatusIssued 3, ProposalStatusRejected 4. Filter names PENDING, ISSUED, REJECTED, ALL.
- Limit: PageSize x PageNo <= WorklistMaxRows (1,000) and at most 100 hits per product per query. WorklistPageSize (20) is the default page size; the portal asks for 15.
- Fields returned per row: ProposalNo, PolicyNo, ProductCode, ProductSubcode, ProductName, PlanId, PlanName, InsuredName, Premium, EffectiveDate, ProposalDate, Status, StatusCode, RejectReason (ProposalRejectDesc), Maker, MakerId, CarrierCode, CarrierName.

Cancellation queue (type CANCELLATION)

- UPWorklist.endoSearch calls the endorsement search POST /platform/endo/v1/query through UPPlatformClient (the same search service, endorsement index) with EndoType 3 (cancellation) and EndoStatus.
- EndoStatusPending 120 is a pending request, EndoStatusIssued 300 an approved cancellation, EndoStatusWithdrawn 600 a withdrawn one. A rejected request and a request replaced when an approval runs are both 600, so only PENDING and ISSUED can be listed.
- Rows add RequestNo (EndoNo), RequestStatus, RequestedOn and CancellationDate. Detail of one request is UPCancelDetail: it finds the pending endorsement with the signed policy id (as pendingCancellation) and returns reason (OtherCancelReason), requester (DataEntryUserId mapped to a name), the refund calculation and CanDecide.

15.5 Dashboard

UPDashboard has two sections so that the portal can run them in parallel. SUMMARY returns proposal counts (pending, issued, rejected, submitted, pending older than a day), cancellation counts and a per-product split: per product three policy searches (one per status), one for pending older than a day and two endorsement searches. COMMISSION takes one ProductCode, searches the issued policies by EffectiveDate, loads up to DashboardMaxPolicies of them and returns premium, commission, policies in force and cancelled (a policy whose status is not PolicyStatusInforce counts as cancelled), a by-plan table and the flags IssuedInWindow, Counted, Truncated. The portal calls SUMMARY once and COMMISSION for each of the 7 products at the same time (8 calls).

- Window: FromDate / ToDate, default the last 30 days, at most DashboardMaxDays (180) wide; otherwise 400.
- Scope: ALL or MINE. A caller who holds only the Maker role is always restricted to their own submissions (AgentCode condition).
- Cost: roughly 7 to 10 seconds per call in MC (about 40 platform calls for SUMMARY; one load per policy for COMMISSION).
- All figures include test data created in MC.

15.6 Maker-Checker Rule and the Test Switch

With RoleEnforcement = Y and AllowSelfApproval = N a user can never approve, reject or decide something they created. The live role check (role_check.py) proves this by switching UP_UserRole rows on and off for the single test account. With AllowSelfApproval = Y the rule is skipped, which is the state of MC so that one person can test the whole flow. The portal reads the switch from /up/me (AllowSelfApproval) and disables the approve and reject buttons only when the rule would refuse them.

15.7 New API Contracts (examples)

All requests are POST with a JSON body and the bearer token unless noted; ChannelCode is S0HAR. Errors use the standard body (section 9). The examples are shortened from real MC responses.

GET /up/me

```
{ "UserName": "ravi.teja@insuremo.com", "UserId": "1188178494", "DisplayName": "Ravi Teja (test user - replace with bank staff)",
  "Email": "ravi.teja@insuremo.com", "Branch": "001",
  "Roles": ["MAKER", "PROPOSAL_CHECKER", "CANCELLATION_CHECKER"],
  "RoleEnforcement": "Y", "AllowSelfApproval": "Y" }
```

POST /up/worklist

```
request : { "ChannelCode": "SOHAR", "Type": "PROPOSAL", "Status": "PENDING", "ProductCode": "HC001",
"FromDate": "2026-09-03", "ToDate": "2026-10-03", "PageNo": 1, "PageSize": 15 }
response: { "Type": "PROPOSAL", "Status": "PENDING", "PageNo": 1, "PageSize": 15, "Total": 117,
  "Records": [ { "ProposalNo": "PHC0010000000243", "PolicyNo": "", "ProductCode": "HC001", "ProductName":
"Hospital Cash Benefit",
  "PlanId": "1", "PlanName": "Plan 1", "InsuredName": "Workflow Test", "Premium": 70, "Maker": "Ravi Teja
(...)", "MakerId": "1188178494",
  "ProposalDate": "2026-10-03", "Status": "Bound", "StatusCode": "2", "RejectReason": "", "CarrierCode":
"AFIC", "CarrierName": "Afillar Insurance Company" } ] }
```

POST /up/dashboard (SUMMARY and COMMISSION)

```
request : { "ChannelCode": "SOHAR", "Section": "SUMMARY", "FromDate": "2026-09-03", "ToDate": "2026-10-03",
"Scope": "ALL" }
response: { "Proposals": { "Pending": 567, "PendingOlderThanOneDay": 563, "Issued": 429, "Rejected": 332,
"Submitted": 1328 },
  "Cancellations": { "Pending": 110, "Issued": 199 }, "ByProduct": { "HC001": { "Pending": 117, "Issued": 74,
"Rejected": 51 } },
  "FromDate": "2026-09-03", "ToDate": "2026-10-03", "Scope": "ALL" }
request : { ..., "Section": "COMMISSION", "ProductCode": "HC001" }
response: { "ProductCode": "HC001", "IssuedInWindow": 69, "Counted": 47, "Truncated": true, "InForce": 16,
"Cancelled": 31,
  "Premium": 3080, "Commission": 2156, "CancelledPremium": 6510, "CancelledCommission": 4557,
  "ByPlan": [ { "PlanName": "Plan 1", "Policies": 8, "Premium": 560, "Commission": 392 } ] }
```

POST /up/cancel-detail

```
request : { "ChannelCode": "SOHAR", "PolicyNo": "POUPPA00100000072" }
response: { "RequestNo": "POUPPA00100000072-001", "RequestStatus": "PENDING", "CancelReason": "spike",
"CancellationDate": "2026-10-04",
  "RequestedBy": "Ravi Teja (...)", "RequestedOn": "2026-10-03", "CanDecide": "Y", "CarrierCode": "AFIC",
"CarrierName": "Afillar Insurance Company",
  "PolicyInfo": { "PolicyNo": "...", "InsuredName": "...", "EffectiveDate": "...", "ExpiryDate": "..." },
"PolicyStatus": { "Code": "2", "Description": "Effective" },
  "PremiumDetails": { "Premium": 68, "FreeLookDays": 15, "DaysSinceCommencement": 0, "ProRataPremium": 68 },
  "RefundDetails": { "CancellationType": "FREE_LOOK", "RefundPercent": 100, "RefundPremium": 68 } }
```

POST /up/share

```
request : { "ChannelCode": "SOHAR", "DocType": "POLICY", "Channel": "BOTH", "PolicyNo": "POHC00100000068",
"Mobile": "96891234567" }
  (DocType QUOTATION needs "Details": { InsuredName, ProductName, PlanName, Premium, EffectiveDate }
plus ProductCode / ProductSubcode and Email or Mobile)
response: { "DocType": "POLICY", "Channel": "BOTH", "Notifications": [ { "Event": "SHARE_POLICY", "Recipient":
"CUSTOMER", "Channel": "EMAIL", "To": "M***@gmail.com", "Status": "DRYRUN" },
  { "Event": "SHARE_POLICY", "Recipient": "CUSTOMER", "Channel": "SMS", "To": "****4567", "Status": "DRYRUN" } ]
}
```

Existing responses gained one additive field, *Notifications[]*, on *UPProposal (create)*, *UPIssue*, *UPProposalReject*, *UPCancelCheck* (only when a new request is created) and *UPCancelApprove*. A 403 body has `status 403` and `code UP-403`.

16. Notifications (Email and SMS)

16.1 Design

`UPNotify.fire(event, params, customer, makerUserId, onlyChannel)` reads every active row of `UP_NotifyTemplate` for the event, resolves the recipients and sends or records each message. It never throws: a problem becomes a result row, so a failed alert can not fail a proposal, an issue or a cancellation. The caller adds the result list to the response as `Notifications[]`. Recipients: a role (all active holders in `UP_UserRole`), `MAKER` (the creator of the item, looked up by platform user id) or `CUSTOMER` (the Email and Mobile of the customer, taken from the stored proposal, unmasked, or from the request of `UPShare`).

16.2 Events and Templates

Event	Recipient (email and SMS each)	Sent by
SUBMITTED	Proposal Checkers	UPProposal after a successful create
PROPOSAL_ISSUED	Maker, customer	UPIssue
PROPOSAL_REJECTED	Maker	UPProposalReject (placeholder {Reason})
CANCEL_REQUESTED	Cancellation Checkers	UPCancelCheck, only when a new request is created
CANCEL_APPROVED	Maker, customer	UPCancelApprove (placeholder {Refund})
CANCEL_REJECTED	Maker	UPCancelApprove with REJECT
SHARE_QUOTATION, SHARE_PROPOSAL, SHARE_POLICY	Customer	UPShare (event name from the keys <code>ShareDocQuotation</code> , <code>ShareDocProposal</code> , <code>ShareDocPolicy</code>)

Placeholders (`{Name}`; a placeholder without a value becomes empty): `ProposalNo`, `PolicyNo`, `Insured`, `Product`, `Plan`, `Premium`, `EffectiveDate`, `Link` (`PortalUrl`), `Insurer`, `Checker`, `Maker` (both are the calling user's display name), `RequestNo`, `Reason`, `Refund`. `UPNotify.paramsOf(policy, extra)` builds them. A template row is identified by `TemplateKey = Event|Recipient|Channel` and has `Subject`, `Body`, `SmsTemplateCode` and `IsActive`.

16.3 Modes and Result Status

Status	Meaning
SENT	The SNS call was made (NotifyMode LIVE).
DRYRUN	NotifyMode is not LIVE: nothing was sent. This is the state of MC.
NO_ADDRESS	No email or mobile known for that recipient (for example the test user has no mobile).
NO_SMS_TEMPLATE	LIVE mode, SMS, and <code>SmsTemplateCode</code> is NONE.
FAILED	The SNS call raised an exception (logged as <code>notification failed</code>).

The `To` value in a result is masked (`r*@insuremo.com, *4567`).

16.4 SNS Integration Details

- Email: `SnsSdkClient.emailApi().newSendEmailRequestBuilder().requestBody(SendEmailRequest)`. Model fields (found by reflection in MC): `accountName`, `alias`, `attachments`, `bcc`, `cc`, `content`, `contentAlternative`, `contentType`, `customHeaders`, `replyTo`, `senderParams`, `subject`, `templateCode`, `templateParams`, `to`. The code fills `to`, `subject`, `content` and `accountName` (when `EmailAccountName` is not NONE).
- SMS: `smsApi().newSendSmsRequestBuilder().requestBody(SendSMSRequest)`. Model fields: `accountName`, `autoAddSign`, `senderParams`, `signName`, `templateCode`, `templateParams`, `to`, `useZhBracket`. **There is no free-text field**: the SMS is the SNS template named by `templateCode` filled with `templateParams`. The text in `UP_NotifyTemplate.Body` for SMS is therefore documentation and dry-run preview only; the real text lives in the SNS SMS template.
- The request model is built with `IComposerJsonUtils.mapToJsonObject(map, Class)` from the Java field names. Whether the SNS service accepts exactly these names at runtime is unproven.

- MC findings: 0 email accounts and 0 SMS accounts. Email senders offered by the tenant: SENDGRID, SMTP, AWS PINPOINT, AWS SES, Microsoft Graph. SMS senders include AWS SNS, AWS End User Messaging, ALI, OneWay, EngageLab, worldSMS and several country specific ones.

16.5 Switching to Live Delivery

1. A tenant administrator creates an email account and an SMS account in the SNS service and, for SMS, one SMS template per message (the template parameters are the placeholder names above).
2. Put the account names in `EmailAccountName` / `SmsAccountName` (NONE means the default account) and each SMS template code in the `SmsTemplateCode` column of the SMS rows of `UP_NotifyTemplate` (record-save).
3. Check that every staff row in `UP_UserRole` has a real Email and Mobile.
4. Set `NotifyMode = LIVE` (`python3 setcfg.py NotifyMode LIVE`).
5. Exercise each event once (`role_check.py` produces all of them) and read the platform log for `notification failed` lines. Expect to adjust field names or template parameters on the first run.

17. The Portal (UIC Page)

17.1 What and Where

Item	Value
Page	Sohar Insurance Portal, path <code>sohar-insurance-portal</code> , page id 894239494239121408, tenant <code>uponly</code> , environment <code>portal</code>
URL	<code>https://portal.insuremo.com/uic-web/#/p/sohar-insurance-portal</code> (a one-time login-free link comes from <code>imo uic page exchange-code --path sohar-insurance-portal</code>)
Source	<code>uic_pages/uponly/portal/sohar-insurance-portal/index.html</code> (one self-contained HTML file with inline CSS and JavaScript; copy in <code>deliverables/Sohar_Insurance_Portal_index.html</code> ; also <code>metadata.json</code> , <code>prompt.md</code>)
Template	standard UIC template (vanilla HTML and JavaScript, no build step, no external libraries; the charts are inline SVG)
Size	about 750 lines; section 17.2 lists the functions

View (role)	Screen	APIs called
Dashboard (all)	Tiles, three charts, tables by product and plan; period and scope filter	<code>/up/dashboard</code> (SUMMARY + 7 x COMMISSION)
New quotation (Maker)	Step 1 quotation (basic details, plan cards, share), step 2 proposal details, step 3 submitted	<code>/up/get-plans</code> , <code>/up/share</code> , <code>/up/proposal</code> , <code>/up/doc_generation</code>
My submissions (Maker)	Queue MINE with filters; detail with reject reason and PDF / share	<code>/up/worklist</code> (MINE), <code>/up/load-policy</code>
Proposal approvals (Proposal Checker)	Queue PROPOSAL; detail with approve / reject (reason in a dialog), PDF, share	<code>/up/worklist</code> , <code>/up/load-policy</code> , <code>/up/issue</code> , <code>/up/proposal-reject</code> , <code>/up/share</code>
Cancellations (Maker, Cancellation Checker)	Tab Requests (queue and detail with refund calculation and approve / reject), tab Raise a request (Maker)	<code>/up/worklist</code> (CANCELLATION), <code>/up/cancel-detail</code> , <code>/up/cancel-check</code> , <code>/up/cancel-approve</code> , <code>/up/load-policy</code>
Policies (all)	Search by policy or proposal number; certificate / application PDF, commission, share	<code>/up/load-policy</code> , <code>/up/doc_generation</code> , <code>/up/commission-query</code> , <code>/up/share</code>

17.2 Structure of the File

The script is organised in blocks, in this order: UIC helpers (token, tenant, URL), configuration (CFG, ROLE), utilities (api(), error rendering, modal, alerts), state and master data (loadMaster: ProductMaster, ProductRule, PremiumMode, code tables, UP_Carrier through `/up/list-master-table`), navigation by role (NAV, buildNav, go), shared renderers (policySummary, getDoc, shareDialog), the generic Queue class used by three views, proposal and cancellation detail, policies, the quotation wizard, the dashboard (inline SVG helpers barSvg, hbarSvg, donutSvg) and the wire-up (bind, startApp, sign-in / sign-out, init).

Function index of index.html (generated)

Function	Parameters	Line	Comment
getHostname		252	
getTenantCode		253	
buildApiUrl	apiPath	254	
getAccessToken		256	
portalToken		257	
redirectToLogin		258	
apiUrl	path	267	

Function	Parameters	Line	Comment
logCall	method, path, body, status, resp	279	
api	method, path, body, quiet	285	Calls an /up API. Returns {ok,status,data,blob}; never throws; 401 sends the user to login. quiet = no spinner
showAlert	kind, html	300	
errHtml	res, fieldMap	301	
showApiError	res, fieldMap	307	
download	blob, name	308	
openModal	html	309	
closeModal		310	
clearBad	scope	311	
fieldErr	id, msg	312	
notifyHtml	list	314	Short text for the Email / SMS results returned by the APIs.
loadMaster		328	
products		334	
variantsOf	code	335	
buildNav		351	
go	view	360	
statusBadge	r	375	
coverRows	plan	376	
policySummary	p	378	Read-only summary of a loaded proposal / policy.
getDoc	type, no, subcode	387	
shareDialog	ctx	393	Share dialog. ctx = {docType, proposalNo?, policyNo?, productCode?, subcode?, details?, email?, mobile?}
Queue		416	
buildQueues		454	
openProposal	row, mountId, canAct	461	---- proposal detail (Proposal Checker acts, Maker only views) ----
openCancel	row	496	---- cancellation detail (Cancellation Checker acts) ----
cancelTab	t	516	
cancelLoadPolicy		517	
cancelSubmit		524	
doSearch		538	
fillStatic		552	
onProduct		560	
onVariant		561	
termRequired		569	
onMode		570	
expiryOf	effStr, term	572	
ageAt	dobStr, effStr	573	
newStep	n	574	
validateQuote		578	
quoteBody		586	
doQuote		591	

Function	Parameters	Line	Comment
renderPlans	plans	600	
enterProposal		606	
validateProposal		607	
proposalBody		614	
doSubmitProposal		625	
resetWizard		636	
barSvg	rows, w	642	
hbarSvg	rows, w	647	
donutSvg	parts	652	
dashWindow		660	
loadDashboard		665	
bind		699	
startApp		707	
loginMessage	kind, text	724	---- sign-in / sign-out ----
endSession	message	725	
armIdle		727	
doLogin	e	728	
init		742	

17.3 Sign-In and Session

- On load the page shows the sign-in screen. It offers a user name and password form and, when the browser already holds an InsureMO portal session (`sessionStorage.Authorization`), a second button "Continue as the user signed in to InsureMO".
- Sign-in: `POST {CFG.casUrl}/cas/ebao/v2/json/tickets` (same call as step 0 of the Postman collection) with headers `x-mo-user-source-id: platform`, `x-mo-tenant-id: {CFG.tenant}` and body `{username, password}`. The token is taken from the reply (`access_token`, `token`, `ticket`, `Token` or the same under `data`), as the collection's test script does. A failure shows one plain message ("Sign-in failed. Check your user name and password." or "not available") and no server text.
- Storage: the token is kept in `AUTH.token` and, for a reload, in `sessionStorage.sohar_token` (this tab only). The password is never stored or logged. Two modes: `login` (own sign-in) and `session` (the portal session token).
- Sign-out button and idle timeout (`CFG.idleMinutes = 30`) remove the stored token and reload the page. A 401 in `login` mode signs the user out with a message; in `session` mode it redirects to the InsureMO login as the UIC template does.
- After sign-in `/up/me` supplies the roles; no role gives an "Access denied" screen. The menu is only a convenience: every API enforces the role again.

Not verified: a successful sign-in with a real password (no password was available), and whether the CAS ticket service always accepts a browser cross-origin call from the portal host (preflight returned `access-control-allow-origin: *`). Test with a real user first.

17.4 API Access

`apiUrl(path) = buildApiUrl(CFG.routeBase) + path`, where `buildApiUrl` is the standard UIC helper: on a gateway host it uses the path as is, otherwise it prefixes `/api`; `{{tenant_code}}` is replaced (`platform` on the portal host). `CFG.routeBase` is `/{{tenant_code}}/api-orchestration-test/v1/flow`, the same route the collections use as `baseUrl`. All tests replaced the base with a stub through `sessionStorage.sohar_api_base`. That hook is still in the code (`API_OVERRIDE`): remove it before go-live, and confirm the real in-portal URL with a first manual call.

17.5 Configuration and Branding

- CFG (top of the script): `casUrl`, `tenant`, `idleMinutes`, `routeBase`, `channel` (SOHAR), `branch` (001), `userName` (SoharPortal, sent as `UserName` in quotations), `payLG` / `payLC` (payment info codes used by the reference flows), `riskCode` (R00001), `pageSize` (15).
- Products, variants, premium modes, age and term limits, free look and refund percent are read at start from the master data API (`UP_ProductMaster`, `UP_ProductRule`); nothing about a product is hard coded except `FEMALE_ONLY` = [FP001] (Female Protection is sold to women only).
- Marital status shows only the codes 1 and 2 of `InsMaritalStatus` because that code table also holds salutation values.
- Brand colours, font and chart colours are CSS variables in one `:root` block at the top (approximate Sohar International Bank public colours; the official guideline was not available). The logo is a text badge.

17.6 Deploy and Test the Portal

Task	Command / script
Deploy a change	<code>imo uic page push --file uic_pages/uponly/portal/sohar-insurance-portal/index.html --memo "... " --profile portal:uponly</code> (pull first if someone else may have changed the page: <code>imo uic page pull --id 894239494239121408</code>)
Get a login-free link	<code>imo uic page exchange-code --path sohar-insurance-portal --profile portal:uponly --json</code> (single use)
Version history / rollback	<code>imo uic page history list --id 894239494239121408</code> , <code>imo uic page history rollback --page-id ... --version ...</code>
Browser tests (headless Chromium, stub gateway)	<code>flow2.js</code> (23 checks: dashboard, quote, share, submit, queue, approve, reject, cancellation, policy view), <code>login.js</code> (11 checks: sign-in screen), <code>run.js</code> (12-step flow per product); run with <code>node <script> <outputDir> <ProductCode></code> ; screenshots in <code>deliverables/ui_test_workflow</code> , <code>ui_test_login</code> , <code>ui_test</code>
Role / notification API checks	<code>python3 role_check.py</code> (35 checks; toggles <code>UP_UserRole</code> rows and <code>AllowSelfApproval</code> , restores them at the end)

The browser tests need a valid `imo` token (they read it with `imo auth prepare`) and Playwright with Chromium. They talk to the real MC APIs and therefore create test proposals, policies and cancellations each time.

17.7 Portal Limitations

- A submitted proposal cannot be edited in the portal (the API supports it for the creating maker); a rejected proposal is resubmitted as a new quotation.
- The proposal detail shows what the platform returns: the mobile is masked by the platform; the ID number is masked by the page (last four characters).
- Queues show up to 15 rows per page; all totals include test data.
- The dashboard needs 7 to 10 seconds and is capped as described in 15.5.
- No audit trail beyond the platform's own records; no export.

18. PA001, iHub and the Other Assets in the Tenant

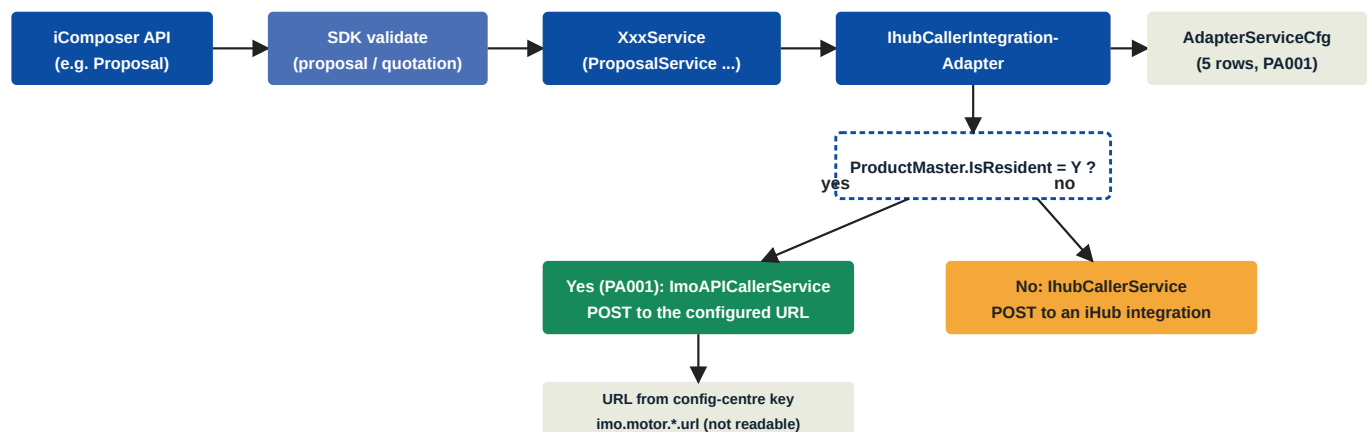
This section records what was found about the existing PA001 solution so that nobody assumes the new products work the same way. It is based on a read-only copy of the tenant's iComposer code and tables taken earlier in this project. A second, live check was not possible when this guide was written because the `imo` login had expired, so verify anything critical in the tenant.

18.1 What Exists

Group (iComposer)	Content	Notes
Policy_API (group 572066178)	13 APIs: GetPlans, Quote, Proposal, UpdateProposal, IssuePolicyAPI, ProposalReject, LoadPolicyData, DocGeneration, ApplicationDocGeneration, FeedFile, QuickProposal, CreateUpdateProposal, CreateProposalReject. 8 functions: ProposalService, UpdateProposalService, IssueService, ProposalRejectService, QuoteService, ImoAPICallerService, APICallerUtils, CreateUpdateProposalBusinessFunction.	The PA001 flow of the reference collection (routes <code>/get-plans</code> , <code>/proposal</code> , ...).
POLICY_API (group 764483681)	17 APIs (quote, policy, claims, document and master-table APIs) and 33 functions, including the adapter functions <code>IhubCallerIntegrationAdapter</code> , <code>IhubTransformerCallerIntegrationAdapter</code> , <code>IhubCallerService</code> , <code>RestClientService</code> , <code>DataTableUtil</code> , <code>ExceptionHandler</code> , <code>ErrorCodeEnum</code> .	Shared building blocks of the PA001 design.
UP_PRODUCTS_API (group 1188491119)	Everything described in this guide.	Created for the new products.
Platform AI group (706710450)	About 45 <code>AI_*</code> APIs supplied by the platform.	Not ours; do not change.

18.2 How the PA001 APIs Work

PA001 APIs (group Policy_API, routes `/get-plans`, `/proposal` ...)



New products (group UP_PRODUCTS_API, routes `/up/...`)



Figure 5: PA001 call pattern (top) compared with the new products (bottom)

1. The API validates the request with the platform SDK (`proposalApi().newValidateRequestBuilder()` or the quotation equivalent).
2. A service function (for example `ProposalService.getCarrierProposal`) takes the provider code (`CarrierCode`, default AFIC), the country (OM) and the transaction type code (GETPLANS, PROPOSAL, ISSUEPOLICY, UPDATEPROPOSAL, PROPOSALREJECT) and looks the numeric transaction type up in the iTable `AdapterServiceCfg` (5 rows for PA001: 10001 to 10005).
3. `IhubCallerIntegrationAdapter.callIntegrationAdapter` builds an `IComposerIntegrationAdapterRequest` and calls the platform's integration adapter service. The adapter row names an implementation

(`iComposer.IhubCallerIntegrationAdapter`) and a URL setting (`imo.motor.quote.url`, `imo.motor.proposal.url`, `imo.motor.issuepolicy.url`, `imo.motor.updateproposal.url`, `imo.motor.proposalreject.url`) that is stored in the platform's config centre.

4. `sendRequest` then branches on `ProductMaster.IsResident` of the product: **Y** posts to the URL through `ImoAPICallerService` (an InsureMO API, no iHub); otherwise `IhubCallerService.executeIntegration` posts to an iHub integration and turns an iHub error into `IHUB_SERVICE_CALL_ERROR` / `IHUB_SERVICE_INTERNAL_ERROR`. For non-resident products the answer is also run through `CreateUpdateQuoteBusinessFunction`.
5. Both PA001 variants have `IsResident` = **Y** in the product table, so PA001 should take the direct branch.

18.3 Comparison

Topic	PA001 APIs	New /up APIs
iComposer group	<code>Policy_API</code> and <code>POLICY_API</code>	<code>UP_PRODUCTS_API</code>
Routes	<code>/get-plans</code> , <code>/proposal</code> , <code>/issue</code> ...	<code>/up/get-plans</code> , <code>/up/proposal</code> , <code>/up/issue</code> ...
Product data tables	<code>ProductMaster</code> , <code>ProductPlanRelation</code> , <code>DocTemplate</code> , <code>AdapterServiceCfg</code>	<code>UP_ProductMaster</code> , <code>UP_ProductPlanRelation</code> , <code>UP_DocTemplate</code> , ... (<code>UPDataUtil</code> maps the logical names)
Validation	Platform SDK validate	<code>UPValidator</code> with <code>UP_FieldRule</code> rows (plus the platform's own checks on save and issue)
Backend call	Integration adapter, URL from the config centre, direct IMO call for resident products, iHub for others	Platform SDK and endorsement REST called directly with the caller's token
iHub	Available through the adapter (not used for resident products)	Not used
Settings	Config centre keys and <code>AdapterServiceCfg</code>	<code>UP_ApiConfig</code> (a missing key is <code>UP-CONFIG</code>)
Errors	<code>ExceptionHandler</code> , <code>ErrorCodeEnum</code>	<code>UPErrorHandler</code> (400 / 403 / 404 / 409 / 500 contract)

18.4 What Is Not Known

- Where the config-centre URLs `imo.motor.*.url` point (they are not readable with the tools used), and therefore what the PA001 "IMO API" branch calls.
- Whether any iHub integration, validator, transformer or Rego rule exists for PA001 or any other product in the tenant (the `imo ihub integration search` command was not run successfully).
- Whether the code copy used for this analysis still matches the tenant (taken earlier in the project).
- How PA001 behaves in UAT; the UAT plan lists its dependencies.

To close these points: log in again (`imo auth login --profile portal:uponly`), run `imo ihub integration search --scope own --profile portal:uponly --json`, reload the `Policy_API` and `POLICY_API` groups into a scratch workspace (`imo icomposer init --group-id <id>`) and read the config centre values with a platform administrator. Do not change PA001: run `snapshot.py` before and after around any work in the same tenant.

18.5 If iHub Is Wanted Later

A bank or carrier integration for the new products would be a new function beside `UPPlatformClient` that posts the request to an iHub integration URL held in `UP_ApiConfig` (with the allowed host list applied as `requireAllowedHost` already does), a new key per transaction type, and a switch per product in `UP_ProductRule`. The PA001 adapter (`IhubCallerIntegrationAdapter` / `AdapterServiceCfg`) can be studied but should not be shared: it is wired to PA001's rows and to `ProductMaster`.

19. Operations Runbook and Handover Checklist

19.1 Switches and How to Change Them

Key (UP_ApiConfig)	MC now	Meaning	Go-live value
RoleEnforcement	Y	Role checks active. N = every API open as before v1.2.	Y
AllowSelfApproval	Y	Creator may approve own item (single test account).	N
NotifyMode	DRYRUN	LIVE sends through SNS, anything else records only.	LIVE after accounts exist
EmailAccountName / SmsAccountName	NONE	SNS account names (NONE = default account).	per tenant
PortalUrl	https://portal.insuremo.com/uic-web/#/p/sohar-insurance-portal	Link placed in messages.	per environment
DashboardMaxDays, DashboardMaxPolicies	180, 60	Window and per-product policy cap.	tune
WorklistPageSize, WorklistMaxRows	20, 1000	Default page size; index limit.	keep
EndoSearchPath, SearchPath, EndoSearchModule	set	Platform search routes used by the queues.	check per tenant

Change a key: `python3 setcfg.py Key Value [Key Value ...]` (it reads `UP_ApiConfig` with `record-list`, saves the row with `record-save` and prints the old value). Changes apply from the next request; no deployment is needed.

19.2 Users and Roles

1. A platform administrator creates the person's InsureMO user in the tenant (this solution cannot create users or passwords).
2. Add one row per role to `UP_UserRole` (`record-batch-save`; `UserRoleKey = user|ROLE` must be unique, `Id` unique): `UserName` (as the platform reports it, usually the email), `UserId` (platform user id, needed for alerts to the maker), `Role`, `DisplayName`, `Branch`, `Email`, `Mobile`, `IsActive` Y.
3. To remove access set `IsActive` to N (`record-save`) rather than deleting the row.
4. Check with the user's own login: the portal header shows name and roles; `GET /up/me` returns the same.

`workflow_setup.py` shows the row format. Re-running the setup scripts is not idempotent for rows: the platform refuses a duplicate `Id`.

19.3 Troubleshooting the v1.2 Features

Symptom	Likely cause	Check
403 UP-403 for everything	User not in <code>UP_UserRole</code> or the row is inactive	<code>GET /up/me</code> ; the table rows
403 on approve / reject	Maker-checker rule (creator = caller) with <code>AllowSelfApproval = N</code>	<code>cancel-detail</code> shows <code>CanDecide</code> ; use another checker
Queue is empty but proposals exist	Date window, product filter, or search index delay of a few seconds after a change	Retry; check <code>ProposalStatus</code> of the proposal
"My submissions" misses old proposals	Created before v1.2 (no <code>AgentCode</code>)	Expected
Dashboard slow or truncated	Cap <code>DashboardMaxPolicies</code> ; many policies	Narrow the window; raise the cap carefully
Notifications show DRYRUN	<code>NotifyMode</code> is not LIVE	Section 16.5
N0_ADDRESS for the maker	The maker has no row (<code>UserId</code>) or no Email in <code>UP_UserRole</code>	Add the user row
Portal: "Access denied"	User has no role	Add roles (19.2)

Symptom	Likely cause	Check
Portal: first call returns 404	Route prefix of the real portal frame	Section 17.4; browser network tab
Sign-in always fails	Wrong credentials, user in another tenant, or reply field name different	Browser network tab for the ticket call; compare with Postman step 0
Push fails with token is invalid or expired	imo login expired (24 hours)	imo auth login --profile portal:uponly

19.4 Go-Live Checklist for the v1.2 Additions

1. Replace the test rows of `UP_UserRole` with bank staff; set `AllowSelfApproval = N`; run `role_check.py` (it flips the switch temporarily: run it in a test tenant, or accept the brief change).
2. Create SNS email and SMS accounts and templates; fill the account keys and `SmsTemplateCode`; set `NotifyMode = LIVE`; send one test of every event.
3. Delete the probe API `UPSpokeWho`; remove the `sohar_api_base` hook (`API_OVERRIDE`) from the portal; confirm the in-portal route with a manual call.
4. Verify a real password sign-in (field name of the CAS reply).
5. Decide about role checks on `UPFeedFile` and `UPListMasterTable`; decide about masking the ID number and the platform `TempData` block.
6. Update the specification, UAT plan and checklist, policy model, OpenAPI and Postman collection for the new APIs, tables and keys.
7. Re-run the full suites: `run_tests.py`, `carrier_check.py`, `role_check.py`, the three browser scripts, `security_probe.py`, `snapshot.py` before / after.
8. Decide what to do with the test data in the tenant (it cannot be deleted through the APIs).

19.5 What a New Developer Needs

Need	Detail
Access	An InsureMO user in the MC tenant with iComposer, iTables, product configuration and UIC rights; the imo CLI logged in (<code>imo auth login --profile portal:uponly</code>). No passwords or tokens are stored in the repository or in this guide.
Repository	Branch <code>claude/wizardly-newton-ivfnle</code> of <code>br8innovates/Tablet</code> . Portal: <code>uic_pages/</code> . Delivered copies and tests: <code>deliverables/</code> (<code>workflow_src/ic_src</code> holds the iComposer source of v1.2 and the set-up and test scripts; <code>ui_test*</code> hold the screenshots). The full build folder (document generators, test suites, samples) is in the package zip.
Order of reading	Section 1.4, then 2, 15, 17, 18; then 4 (code reference) next to the source; then 6 (configuration) and 19.
First actions	<code>GET /up/me</code> ; run <code>role_check.py</code> (several minutes); open the portal and walk the flow in all three views; read <code>UPAccess</code> , <code>UPWorklist</code> , <code>UPNotify</code> (about 450 lines together).

19.6 Backlog (Consolidated)

#	Item	Suggested action
1	Notifications never exercised LIVE	Section 16.5
2	Real password sign-in untested; portal URL in the real frame untested	Manual verification, then remove <code>sohar_api_base</code>
3	No role check on <code>UPFeedFile</code> / <code>UPListMasterTable</code>	Add <code>requireAny()</code> or a checker role
4	Maker id in <code>AgentCode</code>	Dedicated field or index field when the platform allows
5	Dashboard accuracy and speed (cap 60, 7 to 10 s)	Pre-aggregation (batch) or a reporting table if figures must be exact
6	Rejected cancellation requests not listable	Record the decision outside the platform or read endorsement history
7	No audit table (who approved what, when)	Audit table or the platform audit module
8	ID number and <code>TempData</code> in responses	Mask / strip in <code>UPShaper</code>
9	Edit of a submitted proposal in the portal	Add an edit view (fields are masked in the load response: ask for them again)

#	Item	Suggested action
10	Documents lag behind the code	Update specification, plan, model, OpenAPI, collection
11	PA001 / iHub open questions (18.4)	Live check after login
12	Unit tests for pure functions	Data-driven tests

Appendix A. File Inventory

File (under ic/)	Lines	Status
src/dev/Tenant/UP_PRODUCTS_API/api/UPApplicationDocGeneration/UPApplicationDocGeneration.groovy	25	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPCancelApprove/UPCancelApprove.groovy	25	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPCancelCheck/UPCancelCheck.groovy	26	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPCancelDetail/UPCancelDetail.groovy	28	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPCancelPolicy/UPCancelPolicy.groovy	5	inactive
src/dev/Tenant/UP_PRODUCTS_API/api/UPCommissionQuery/UPCommissionQuery.groovy	137	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPDashboard/UPDashboard.groovy	50	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPDocGeneration/UPDocGeneration.groovy	24	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPFeedFile/UPFeedFile.groovy	178	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPGetPlans/UPGetPlans.groovy	105	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPIssue/UPIssue.groovy	58	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPListMasterTable/UPListMasterTable.groovy	37	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPLoadPolicy/UPLoadPolicy.groovy	37	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPMe/UPMe.groovy	13	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPProposal/UPProposal.groovy	44	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPProposalReject/UPProposalReject.groovy	50	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPShare/UPShare.groovy	72	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPSpikeWho/UPSpikeWho.groovy	20	inactive
src/dev/Tenant/UP_PRODUCTS_API/api/UPUpdateProposal/UPUpdateProposal.groovy	62	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPWorklistApi/UPWorklistApi.groovy	35	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPAccess/UPAccess.groovy	124	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPCancelService/UPCancelService.groovy	364	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPDataUtil/UPDataUtil.groovy	235	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPDocBuilder/UPDocBuilder.groovy	195	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPDocService/UPDocService.groovy	105	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPErrorHandler/UPErrorHandler.groovy	104	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPNotify/UPNotify.groovy	166	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPPlatformClient/UPPlatformClient.groovy	49	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPProductData/UPProductData.groovy	213	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPSecurity/UPSecurity.groovy	116	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPShaper/UPShaper.groovy	126	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPValidator/UPValidator.groovy	334	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPWorklist/UPWorklist.groovy	408	active

Appendix B. Table Definitions

UP_ApiConfig (id 1188575596, 113 rows)

Column	Data type	Primary key
Id	String	Y
Api	String	
Scope	String	
Key	String	
Value	String	

UP_FieldRule (id 1188575333, 167 rows)

Column	Data type	Primary key
Id	String	Y
Api	String	
Path	String	
Label	String	
Required	String	
DataType	String	
MinLength	Integer (long)	
MaxLength	Integer (long)	
Pattern	String	
MinValue	Decimal	
MaxValue	Decimal	
CodeTable	String	
CodeColumn	String	
EnumValues	String	
AppliesTo	String	
WhenPath	String	
WhenValue	String	

UP_ProductMaster (id 1188475672, 11 rows)

Column	Data type	Primary key
Id	String	Y
Description	String	
LOB	String	
IMOProductCode	String	
IsRenewable	String	
IsActive	String	
IsFlatRate	String	
Premium	Decimal	
SumInsured	Decimal	
HasPlans	String	
CarrierCode	String	
CarrierProductCode	String	
CarrierProductName	String	

Column	Data type	Primary key
IsResident	String	
BenefitRelationTableName	String	
LobId	Integer (long)	
CarrierName	String	

UP_ProductPlanRelation (id 1188475676, 57 rows)

Column	Data type	Primary key
Id	String	Y
IMOProductCode	String	
IMOPlanCode	String	
IMOPlanName	String	
CarrierCode	String	
CarrierProductCode	String	
CarrierProductName	String	
CarrierPlanCode	String	
CarrierPlanName	String	
IsActive	String	
Premium	Decimal	
SumInsured	Decimal	
Commission	Decimal	
PremiumModeCode	String	

UP_PlanBenefitRelation (id 1188474890, 148 rows)

Column	Data type	Primary key
Id	String	Y
IMOProductCode	String	
IMOPlanCode	String	
CarrierCode	String	
CarrierProductCode	String	
CarrierPlanCode	String	
BenefitCode	String	
BenefitDesc	String	
BenefitLevel	String	
ParentBenefitCode	String	
BenefitLimit	String	
BenefitLimitDesc	String	
IsMainBenefit	String	
IsAdditionalBenefit	String	
Notes	String	
IsApplicable	String	
LimitBasis	String	
MaxDays	Integer (long)	

UP_ProductRule (id 1188475684, 11 rows)

Column	Data type	Primary key
Id	String	Y
ProductCode	String	
ProductSubcode	String	
PremiumModes	String	
MinTerm	Integer (long)	
MaxTerm	Integer (long)	
MinAge	Integer (long)	
MaxAge	Integer (long)	
MaxEntryAge	Integer (long)	
MaxExitAge	Integer (long)	
MandatoryDocuments	String	
SinglePremiumRule	String	
FreeLookDays	Integer (long)	
ProRataRefundPercent	Decimal	

UP_DocTemplate (id 1188475689, 22 rows)

Column	Data type	Primary key
Id	String	Y
DocumentCode	String	
TemplateName	String	
FileName	String	
ProductCode	String	
ProductSubcode	String	
DocType	String	

UP_DocWording (id 1188556760, 27 rows)

Column	Data type	Primary key
Id	String	Y
ModeCode	String	
Key	String	
Value	String	

UP_BenefitText (id 1188575620, 21 rows)

Column	Data type	Primary key
Id	String	Y
ProductCode	String	
BenefitCode	String	
NameEn	String	
NameAr	String	

UP_FeedColumn (id 1188591810, 29 rows)

Column	Data type	Primary key
Id	String	Y
Sequence	Integer (long)	

Column	Data type	Primary key
Heading	String	
Source	String	

UP_Channel (id 1188591925, 1 rows)

Column	Data type	Primary key
Id	String	Y
ChannelCode	String	
Description	String	
IsActive	String	

UP_Carrier (id 1188663015, 1 rows)

Column	Data type	Primary key
CarrierCode	String	Y
CarrierName	String	
IsActive	String	

UP_UserRole (id 1188710773, 3 rows)

Column	Data type	Primary key
UserRoleKey	String	Y
UserName	String	
UserId	String	
Role	String	
DisplayName	String	
Branch	String	
Email	String	
Mobile	String	
IsActive	String	

UP_NotifyTemplate (id 1188710781, 22 rows)

Column	Data type	Primary key
TemplateKey	String	Y
Event	String	
Recipient	String	
Channel	String	
Subject	String	
Body	String	
SmsTemplateCode	String	
IsActive	String	